



AWS Certified DevOps Engineer - Professional (DOP-C02): Configuration Management and Infrastructure as Code

Elastic Beanstalk

Elastic Beanstalk Overview

Hey Cloud Gurus. Welcome to a new section of the course, and in this section of the course, we're going to talk about configuration management and infrastructure as code and some of the services we'll talk about are Elastic Beanstalk, CloudFormation, OpsWorks, Lambda, API Gateway, so this is a really interesting section of the course and features prominently on the DevOps pro certification exam. So in this lesson, we'll take an overview look at Elastic Beanstalk. Alright, so why would we use Elastic Beanstalk? Well, with Elastic Beanstalk, we can quickly deploy our applications without having to worry about the infrastructure on which those applications run. So maybe a software shop wants to use AWS, they don't have a lot of experience with AWS. They don't want to mess around with servers or CloudFormation. In that case, Elastic Beanstalk is a great choice. So we just need to upload our application and then Elastic Beanstalk will handle capacity provisioning, load balancing. Now, we'll see when we get into Elastic Beanstalk, we do have some control over that, but it's not essential that we control it like with CloudFormation. Alright, so we want to use Elastic Beanstalk. First thing we do is create an application, and an application is a collection of components, environments, versions, configuration. Next, we would upload an application version, such as an application source bundle, and think of a war file or a jar file, so we upload this and then Elastic Beanstalk will automatically create the environment, and it will create and configure the AWS resources needed to run our code. Now, we'll see in upcoming lessons, I'll mention this again, we do have some control over that. We can't configure that as background infrastructure resources, but there are default options, we don't have to do any of that. So we upload our application version, our environment is launched. It's a collection of AWS resources running an application version, think of load balancers, autoscaling groups, and we have default settings for that. We can get in and play around with this a little bit if we want to, not as detailed and in-depth as CloudFormation or OpsWorks. And ultimately, when everything is set up, we can manage our environment. Now, if we want to do updates, our flow comes back here, we upload a more recent version, and it flows out and deploys for us. And notice this line is skipping that



environment, our environment is set. So at this point, we upload the original version, the environment is launched, update version, deploy version. At this point, we're strictly dealing with software and really from the whole point we're dealing with software. This box, Elastic Beanstalk is doing that for us. I'll mention the caveat one more time. Yes, we can get in there and we'll see this, we can configure this a bit. It's not as difficult or involved as CloudFormation. So this is really geared towards software developers and some of the languages supported include Java, .NET, PHP, Node.js, Python, Ruby, and Go. Some of the web servers supported include Apache, IIS, Java, NGINX, Passenger, Puma, Tomcat, and Docker. Yes, we can work with containers in Elastic Beanstalk, and we will have a great lesson on that. So in the field or on the certification exam, it's always good to focus on use cases. Let's talk about some use cases and sort of compare to OpsWorks and CloudFormation. But really, Elastic Beanstalk is great if you have minimal time or the personnel to implement solutions in CloudFormation or OpsWorks. It's geared toward developers and it's the simplest to use. It provides the least visibility under the hood, again, there is some, and we'll see that in upcoming lessons, and we can quickly deploy applications in Elastic Beanstalk. Now, when you think about quickly deploying applications, well, hey, that kind of sounds like it'd be great for a test environment, so we can quickly spin up a test environment using Elastic Beanstalk, do our testing, and then just destroy the environment, so it is great for testing scenarios. Now, what are some antipatterns for Elastic Beanstalk? Well, when you want complete control over your resource configurations, you might consider going in a different direction, maybe OpsWorks or CloudFormation. And if you have a lot of dependencies and a complicated deployment, Beanstalk might not be your best choice. So the main difference between CloudFormation and Elastic Beanstalk, CloudFormation deals more with the AWS infrastructure rather than applications, and it's geared towards systems engineers, whereas Elastic Beanstalk more geared toward developers, so know the use cases for Elastic Beanstalk, as well as some of the antipatterns and just understand the differences between Beanstalk and OpsWork, and also, CloudFormation. Now OpsWorks, these keywords, you see Chef and Puppet in a scenario, you're automatically thinking OpsWorks. Now OpsWorks is great for hybrid environments, so if we have some servers on-prem and that server node is reachable by Chef or Puppet, our on-prem server can be registered with OpsWorks, so it's pretty straightforward. With all that said, let's just go out and look at Elastic Beanstalk in the management console and ingrain some of these concepts. Alright, we're out at the Management Console in Elastic Beanstalk, and again, the premise is we're a small software shop, we don't know much about AWS, just getting into it. We want to deploy our software applications, and as it says, Elastic Beanstalk easy-to-use service for deploying and scaling applications, and down here, you simply upload your code and Elastic Beanstalk automatically handles the deployment from capacity provisioning load balancing and autoscaling. Well, that sounds promising. Let's create an application. So we configure the environment, web server or worker environment, keep it simple,



give our app a name, notice that it autofills the environment and name, choose a platform. Note all of our options here. I'll go with PHP. It's a web application, and for demo purposes, I'll go with the sample application provided. Notice, of course, high availability. We're starting to think about autoscaling and different availability zones. Let's click Next. Configure service access. Alright, it gives us a service role, an instance profile for EC2. I'm not going to create a key pair. Let's click Next. Watch your environment in a custom VPC, instead of the default VPC, so we can create a custom VPC, public IP address, well, let's say it's a website, we'll go activate that, subnets. We could add a database. Upcoming lesson, I'll talk about a database within our Elastic Beanstalk environment and a database outside, pros and cons of that. Think about that. Click Next. And the point here is that, hey, we don't need to know a ton about AWS infrastructure to get up and running. Configure instance traffic and scaling, EC2 security groups. I'll just go default. Well, we went single instance, and we can change that if we'd like to a load balanced environment, but again, there's no obligation to change that. Next. Alright, so we get into monitoring and logging. Enhanced health reporting provides free real-time application operating system monitoring of instances and other resources in your environments. Well, hey, it's free. That's a good deal. And we can configure that if we'd like, but again, sticking with the use case, hey, we're just trying to deploy our application quickly, rolling updates and deployments, so there's a lot of detail we can get into, and there's a lot we can ignore. Click Next, and then we review our environment. Well, we didn't do a whole lot, but it's here for review. Submit. So within minutes, we're deploying our application. Notice on our left-hand pane, a lot of the things we ran through in the setup, notice our events. Our environment is starting, create a security group, create a VPC, waiting for EC2 to launch. This looks kind of familiar. This kind of looks like CloudFormation. Well, guess what? Underneath the hood, there is CloudFormation. To manage these resources and configure them, of course. Well, maybe we want to start thinking about CloudFormation, but hey, again, small software shop, we just want to deploy our applications. This is a great starting point. One final check, click on this domain, there's our application. So that's an overview look at pulling applications using Elastic Beanstalk and building block for several more lessons on Elastic Beanstalk. Now we've talked about Elastic Beanstalk and CloudFormation and understanding use cases and which to use, but also remember that, hey, they can be used together as well. So CloudFormation has resource definitions for both OpsWorks and Elastic Beanstalk. So in our CloudFormation stacks, we can deploy OpsWorks, Elastic Beanstalk, so I talked about that transition between Elastic Beanstalk and CloudFormation, and this is sort of the best of both worlds. We can deploy our resources using CloudFormation and still use Elastic Beanstalk to deploy our applications and they each have their different use cases. Again, OpsWorks is great if you have on-premises nodes that are reachable by Chef or Puppet, and you're good with OpsWorks. So in this lesson, we looked at Elastic Beanstalk and the simplicity of deploying applications using Elastic Beanstalk, and to end, let's just review a couple of tips from the lesson. Again, understand these differences



between Elastic Beanstalk, OpsWorks, and CloudFormation, but also realize they can work together, and again, Elastic Beanstalk and CloudFormation are often paired together. Manage our resources with CloudFormation, but also deploy our applications with Elastic Beanstalk. So our CloudFormation stack can contain Elastic Beanstalk. So I'll leave it at that, and we'll go deeper into Elastic Beanstalk in the next few lessons. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Elastic Beanstalk Deployment Strategies

Hey Cloud Gurus, welcome. In this lesson, we're going to talk about Elastic Beanstalk deployment strategies. Now, before we talk about deployment strategies, let's talk about environment types, and our understanding of environment types will help our overall understanding of deployment strategies. So with Elastic Beanstalk, we have two environment types, a single instance and high availability. So our single instance environment type is pretty self-explanatory. We're going to have a single instance within our Elastic Beanstalk environment. Now, a key point about this single instance is the fact that it has an Elastic IP address, so our instance has an IP address. Why? Well, notice we have an autoscaling group, so with a single instance environment, this in a sense, is a way for us to keep high availability of our instance. So the autoscaling in group will be configured to always keep one instance running, and that's why we have the Elastic IP address. If our instance goes down, well, guess what, we can reuse that Elastic IP address. Autoscaling group will spin up another instance. Now for high availability, well, it's what we would expect. We have a load balancer, we have an autoscaling group, we have a group of instances, so our autoscaling group handles the launching, configuring, and terminating of our instances. And of course, implicitly, we're dealing with more than a single instance. So those are the two environment types, and these deployment types are important for the exam to know these and understand what they're doing. We have all at once, rolling, rolling with additional batch, immutable, and blue/green, and first, we have all at once. So in this diagram, I'm just generically showing three instances and our three instances have version 1. We want to update to version 2, and as its name implies, we're going to perform this update on all of these instances at the same time, and that's an in-place deployment. So in-place means that our infrastructure is not changing at all. So when you think about it, what are the pros of this method? It's going to be fast, and remember again, nothing at all changes, so there are no DNS changes necessary. If something goes wrong, well, something just went wrong to all of our instances, so it's going to complicate rollback a little bit. When that happens, we have to redeploy the old version of our application, so there's some potential more downtime. So start thinking about requirements and use cases, but if you have a requirement where you can't have any downtime, you do not want to use all at once. So to summarize, deploying all at once is fast and requires no DNS changes,



but if there is a problem, you will incur downtime. Alright, now, speaking of no downtime, let's talk about rolling deployments. So with a rolling deployment, and in this case, I'm showing four instances, a rolling deployment is going to guarantee that at least one of our instances is up at all times. So in this situation, just going by the colors, the color coding, we have four instances, we deploy to two of those instances at first, then, we deploy to the other two and that number could be, we could just be deploying to one, but the guarantee is that at least one of these instances is going to be up at all times. Now, the mechanics of this update is that when we deploy to these two instances, they will be detached from the load balancer, so they're not even available to accept traffic. Once the update is completed, they're reattached to the load balancer. Now, what do load balancers do when we attach instances to them? They are going to perform a health check. So that's a little bit of overhead, just keep that in mind.

What happens if those load balancer health checks fail? And that is one of the cons of this deployment method. If it fails halfway through, we could have instances with different versions of the application. And secondly, obviously, if the update fails, we could have some capacity issues where basically our capacity is cut in half. So let's say we're going through with this deployment, and we choose these two instances. At first, we're going to deploy version 2, that deployment fails. Now, remember, we're in an autoscaling group, so what do we do because our environment basically at half capacity and the running instances have version 1. What we do is delete these instances. That autoscaling group will spin up to get back to capacity these instances, and it will spin up the last good version, which would be this version 1. So really, the drawback with rolling deployments is if it fails halfway through, it gets a little bit complicated, and of course, the plus side, we always have at least one instance running. Alright, let's talk about rolling update with additional batches. Now, a rolling update is eventually going to happen, but what happens initially is the update is performed on new instances, not in our live rotation. So we're actually introducing new instances into our autoscaling group, basically doubling the autoscaling group's capacity. So what's the difference between a rolling update and a rolling update with additional batches? Well, with additional batches, we're not going to run into any capacity issues. That's the pros of this method and certainly the pros of it over a rolling update. Now, what are the cons? Well, it will take longer than a regular rolling update. We're doubling our capacity, we're updating those, and then it becomes getting the new instances into service, getting the old instances out of service, so a little more overhead there. What suits your use cases? What suits your requirements? Once our update is complete on these new instances, again, they can be registered with load balancer, load balancer performs the health checks, if they pass, they will become in service, and then some of these other older instances can be removed from the autoscaling group. So our use cases for this one is if we can't afford to reduce our capacity, but we're okay with that little bit extra time to perform the update, this is the way to go. And comparing it to rolling again, so what if we perform this update and this update fails? Well, remember our main autoscaling group of



instances has not been touched yet, so there's no rollback problem. All we have to do is figure out what went wrong with our deployment, fix it, and then reschedule our deployment. So like a rolling deployment, rolling deployment with additional batches prevents downtime, it can take longer than a rolling deployment, but the plus side is the rollback is easier. Alright, what about an immutable deployment? So I'm showing an autoscaling group, we have two instances with version 1, then we have another autoscaling group with version 2. So the idea of the immutable is these version 1 instances are not changed, they are replaced. So these new version 2 instances will be introduced to the autoscaling group and simply replace these. So when we deal with deployment, Elastic Beanstalk will launch our new instances, that's going to double our capacity. The new version will be deployed, the instances will be registered with our load balancer, load balancer will perform health checks. What if something goes wrong? Well, no big deal. We haven't touched these ones. So the rollback is pretty easy and it also prevents downtime. Only after our update is performed successfully, will these old instances be terminated? So if our deployment fails, we just terminate this temporary autoscaling group. And then we're back here, figure out what went wrong with our deployment, and try again. So the cons of this deployment method are, of course, we're doubling our number of instances. Now, another thing to think about any time we're doubling our instances, remember our capacity limitations for EC2 instances. So to summarize the immutable deployment, it does prevent downtime and provides a very clean rollback. The cons, it's at the expense of doubling instances for a short period of time, but I emphasize short period of time. So unless you're under really tight cost considerations, this is a really good deployment methodology. And finally, let's talk about blue/green deployments. So this one's interesting. What happens is we clone our entire environment. Now, I'm focusing on the instances, but we actually clone the autoscaling group, and above that, we clone our load balancer, so we deploy our new version to the green environment. Now, remember again, I'm not showing it, but we actually cloned the load balancer, so these instances are already registered with our load balancer, but okay, we went up another level. Well, guess what? We're introducing another level. We're at the DNS level now. So Route 53 would be fronting all of this, so all we do when our deployment is completed, everything is tested and ready, we just swap those Route 53 URLs. This URL comes over here, this becomes our live environment. So the pros of this method are it prevents downtime because it uses new resources instead of in-place resources, and we can test our updates in an isolated environment. Cons of this method? Well, of course, we're doubling our resources including load balancer autoscaling group, and it requires a DNS name change. That's always an important consideration, adds an extra layer of complexity, but certainly blue/green is a very viable option for us. So just to sort of top this off, so we have Route 53. Now with a blue/green deployment, we actually just swap URLs, so it would go from 100 and 0 to 0 and 100. There are other very similar methodologies such as a canary deployment. In that case, we have our new version over here, and we send a limited amount of traffic to this



new version. The canary deployment is sort of a close cousin to the blue/green deployment. I want to touch on that as well, and that can be executed in Route 53. So we send 10% of our traffic, test it out, if we like it, we send more traffic. Test it some more, if that's all good. Well, we can just keep going until we're sure that our green environment is ready to go, and ultimately, send all of our traffic to the green environment, and ultimately, we can destroy the old blue environment. So there's blue/green, a close cousin to that is the canary deployment. Our blue/green deployment prevents downtime and test updates and isolation. Of course, the cons are we double number of instances, and we are dealing with DNS changes. So that's a look at our deployment types, and before we wrap up this lesson, let's review some key facts about our deployment methodologies. So we have all at once impact of a failed deployment is downtime, and we'll see as we show more that this is just showing 1:00, so it's pretty quick. There will be some downtime. There's no DNS change. Rollback process, we redeploy the last good version, and the code is deployed to the existing instances. Rolling. So impact of a failed deployment, whatever instances we deployed to are going to be out of service. It does take a little bit longer, and there are good reasons for that. It's ensuring 0 downtime. There's no DNS change. Rollback process, we redeploy. Rolling with additional batch. Now remember, with the additional batch, we're deploying to new instances, so the impact is minimal if the first batch fails. It does take longer, again, good reason, 0 downtime. Immutable, so we're starting with completely new instances, we're doubling our capacity, and it's going to take a bit of time to do all of this. And finally, blue/green deployments, it's similar to the immutable, but we are cloning our entire environment all the way up to our load balancers, and it just adds another level, where do we control all of this? Route 53, and we perform a URL swap. So for the exam, know these methods and just know the pros and cons of each, and once you look at them and think about them a bit, it, it really makes a lot of sense, and when do you use which is going to depend on your requirements and your use cases, your risk aversion to downtime, and also just considering some additional charges as far as doubling capacity and those types of things. So that's a look at Elastic Beanstalk deployment types. And with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Elastic Beanstalk Environment Configurations

Hey Cloud Gurus, welcome. In this lesson, we're going to talk about Elastic Beanstalk environment configurations. Now, we think of Elastic Beanstalk as, hey, we don't need to know much about AWS, we just want to upload our code and let AWS handle the rest, and that certainly is an option, but if we want to go deeper into our configurations, we certainly can, so we'll talk about that a bit in this lesson. Just start off with an overview look at environment configurations and we'll talk about some of our configuration options. There are a few key points that you really need to know for the exam, and it probably won't go any deeper



than that. And we'll have a demo as well and just to sort of solidify what we've learned, and we'll finish up with some exam tips. So with that said, let's talk about Elastic Beanstalk environment configurations. Alright, let's talk about environment configurations, and we probably all gone in and configured Elastic Beanstalk environments, and typically, what we do is just accept the defaults. AWS will give us a default environment, but what if we want to go deeper and configure our environment? Maybe we need some queues, maybe we need specific settings in our VPC or an autoscaling group. We may have software dependencies that we have to account for or other specific customizations, and I always try to make an analogy that's relatable to us, and the analogy I would make is think about bootstrapping an EC2 instance. We want specific configurations, specific software on there. This is the analogy I want to make. So when we play around with environment configurations, we're in a sense, "bootstrapping" our Elastic Beanstalk environment. One difference I want to point out is that we can make these changes before, during installation, or after installation with these environment configurations. Now the really key takeaways from this lesson regarding environment configurations, where do you do these configurations? You do them in an EB extensions folder and be specific .ebextensions folder, this is a hidden folder, and in that folder, you place configuration files. And sticking with that bootstrapping analogy, I'll stretch that analogy a little bit more. Think about our user data that we bootstrap with, well the config files contain the customized information that dictates how we configure our Elastic Beanstalk environment, but where .ebextensions, that's where we're making our customizations and the what we're placing configuration files in that folder, you should be in pretty good shape on the exam. As far as the content of those config files, well, guess what? It's either YAML or JSON, and we'll take a look at one here a little bit later in the lesson. It's a format, it's going to be easily readable for us because by this time, we've certainly seen JSON or YAML files very much like CloudFormation, and momentarily, we're going to get into the connection with CloudFormation. So if we want to customize our environment with Elastic Beanstalk, ebextensions, and config files, guess what, CloudFormation plays a part in that. It is not commonly featured, but when we create an Elastic Beanstalk environment, underneath that, we are creating a CloudFormation stack, and we'll see that in a minute. So remember, key features .ebextensions, that's where we save our config files. What if we have a configuration that hey, we like this configuration, we want to use it. Well, we can save that configuration and apply it to other configurations. And just remember, we can create an environment in Elastic Beanstalk or have Elastic Beanstalk within a CloudFormation template. Now, I want to make the distinction here. Anytime again, we create an Elastic Beanstalk environment, a CloudFormation stack is created as well. And again, I'll emphasize that in the demo. Alright, so that's how we configure our Elastic Beanstalk environment. What about some of our options? So we can use configuration files, stored in those ebextensions folders, we can have saved configurations. Settings for any options that are not applied directly to the environment can be loaded from a saved



configuration, and we can make direct changes setting specified during a create environment or an updated environment operation. Now, one thing to know about these options is what if we, during a configuration, we're spinning up in an environment, what if we have multiples of these? What is the precedence? So direct changes that we make, we could make those in the console or from the CLI, they have the highest priority. Second in line are the saved configurations, and finally, config files have the lowest priority. With all that said, when speaking of configuration options, personally, I've seen ebextensions on the exam more than some of these other options. So again, with ebextensions, let's just remember the key points again are config files, the location is in this .ebextensions, and remember that can be a hidden file, just of course, you would have created it. Our configuration files have to end in .config, and again, remember these files are just YAML or JSON. Let's go into a demo and just solidify some of this information. Alright, I'll add the AWS Management Console. Let's go to Elastic Beanstalk, and I want to talk about configurations a little bit, so let's go into our just created environment from the last lesson. Now we have three ways to update our environment directly, which I'll show momentarily, saved configurations, and with ebextensions. So, let's click Configuration, service access, network and database, traffic and scaling, monitoring and logging, but notice all of these are editable. On network and database, enable database. I'm not going to go through this, it takes a while to spin up a database, but from a snapshot, could now create a database. The key point I want to make is this is after the initial environment creation. Now it's unlikely that you'd forget to add your database, but I just want to illustrate the point that with direct updates, we can update our environment after its creation. So I'll just cancel out of this, and let's go back to the main page for our environment. So we created this environment. It's having success. We like this setup. We're going to need more of these in the future, and they can be disposable. In other words, maybe it's a test environment that we use for a month and delete, and this is a recurring situation. So we go over to Actions and we like this configuration. We can save it. And although I didn't do anything spectacular with the security, let's just pretend that we did. We'll call it the Hardened Test Environment, give it a nice descriptive comment, and save it. So now we have a saved configuration. Well, what does that mean? Well, let's see. Let's back out. Let's go to Applications. Alright, so we get a phone call, and we are asked to create a test environment using our hardened configuration. So here's the application, here is the environment, and let's click in on the application. We have one environment already, but again, we're asked to create a test environment. We click Saved configurations, and we just saved this earlier in the lesson. Here is our Hardened Test Environment, so we click the checkbox, launch the environment. One got you, we already have this environment name. Let's change that name so that we don't get a conflict. Click Next. Alright, so we know what's going on here. This is the environment we want to use. We don't want to change anything. It's hardened for us, so I'm going to next through, quickly select a VPC, Subnets, Next, Instances, Dependency, Monitoring and logging, Next, and



review. Again, we know what's in there. Let's click Submit. So we're creating this new test environment from that saved configuration. Now, while this environment is spinning up, we've talked about CloudFormation under the hood. Let's check that out. Awseb for Elastic Beanstalk, create in progress. We can click into this stack and watch this stack build. So in a sense, Elastic Beanstalk is a convenient easy-to-use wrapper on top of CloudFormation. We click on Resources, Elastic IP address, Security group. So that's sort of the magic under the hood, so we got to create complete in CloudFormation. Let's go back over to Elastic Beanstalk, waiting for EC2 instances to launch. Alright, successfully launched environment. So this is how we can quickly spin up environments for testing using a saved configuration. Let's talk about a few exam tips. Now remember, Elastic Beanstalk can be a resource in CloudFormation, and therefore, an Elastic Beanstalk environment can be created with a CloudFormation template. Well, we just saw that. All of our environments underneath are going to be on a CloudFormation stack. And certainly remember this, the hidden folder for our config files, ebextensions. We store our config files in ebextensions, that's how we customize our environment. And the other thing just to remember, these config files are in YAML or JSON. Alright, a few key tips. Three ways to create or update EB environments, saved configurations, direct updates, and configuration files. If you know that and our config files go in ebextensions, you'll be in great shape for the exam. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

Elastic Beanstalk with RDS

Hey Cloud Gurus, welcome. So in this lesson, we're going to talk about Elastic Beanstalk with RDS. So as we've seen in the Management Console, Elastic Beanstalk provides support for running RDS instances in your Elastic Beanstalk environment. And in a lot of cases, that is pretty awesome, and it certainly conforms to the one of the primary use cases for Elastic Beanstalk, basically ease of use. Hey, we just have software, we want to run AWS. You can go ahead and worry about the hardware, the scaling, we'll worry about our code. So as we've seen when we create an Elastic Beanstalk environment, we can't create an RDS database within that environment, so let's talk about that and maybe a drawback that you haven't thought of. So let's just say we're a software development team. We want to upload our code to Elastic Beanstalk, and we need an RDS database. So how do we configure this environment? Well, we've seen in the Elastic Beanstalk Management Console, right in the console, we can set up an RDS database within our Elastic Beanstalk environment, and again, that's great in a lot of cases. So let's take this scenario a little further. So we just started out with Elastic Beanstalk. We're not very experienced at all. We create an Elastic Beanstalk environment and within our environment is an RDS database. Now, one of our developers is not totally happy with the Elastic Beanstalk environment we have. The developer is not real sure how to modify this environment, so the



developer decides, well, I'll just delete the environment and create a new environment the way I want it and we'll be all good to go. Well, do you see a potential problem there? Our RDS database is within that environment, so when the environment is deleted, guess what happens to our RDS database, it's probably going to provide our developer who deleted the environment with an uncomfortable day because that RDS database was deleted as well. So that's where it gets a little bit tricky with Elastic Beanstalk. Yes, it's easy to use, and you can certainly bundle everything all within your environment, but just realize that database is going to be tightly coupled to your Elastic Beanstalk environment, and this may be fine for dev and test environments, but what about a prod environment? Do we want to live that close to the edge? Somebody deletes our environment. Our prod database is gone, and sure, there are safeguards we can take. We can certainly create backups, and we should create backups. We can take snapshots. And again, if you're just a software shop, you don't want to deal with any of this, maybe that's the way you go. Maybe you accept the fact that your RDS database is tightly coupled to your Elastic Beanstalk environment. So what do you do? Well, you better back it up and you better back it up regularly and also take snapshots. So if your environment gets deleted, your database is getting deleted, well, you have backups, you can recreate your RDS database. Okay, great, but what if it's a prod environment? So, restoring that database takes a little bit of time. Do we have downtime restrictions? And again, this is where Elastic Beanstalk can get a little bit more involved than maybe on the surface you would expect it to be. There are things we can do, and if it's just not acceptable then hey, we can lose our database. If we lose our environment, we have downtime restrictions. We can't be down too long. What else can we do? So what we can do is decouple our database. We create our RDS database outside of Elastic Beanstalk, and in doing so, we're making it a better design certainly for a production environment. So if we create Elastic Beanstalk environment, we have an RDS database, we can connect our environment to that database, but it is not within the environment, it's loosely coupled. Deleting the Elastic Beanstalk environment will not delete this RDS database. So if our environment gets deleted, either planned or unplanned, okay, no big deal as far as RDS is concerned. Our RDS database remains untouched, so to decouple our database instance from our Elastic Beanstalk environment, we just create the instance in RDS and configure our application to connect to it on launch. Now, what other advantages does this have? Well, this is just an RDS database. Nothing special about it. It is not solely owned by this Elastic Beanstalk environment, so we could connect multiple environments to this database. Maybe a prod environment is connected to this database, maybe a test environment is connected to the database, maybe both are connected to it. Additionally, think about blue/green deployments. So we have our Elastic Beanstalk environment, and we'll call it a blue environment. We create another green environment with our application updates, and we've seen how we can just swap the URL and perform a blue/green deployment. This becomes simplified when we're not tightly coupling our RDS database within our environment, so this is a great



pattern we can use for RDS working with Elastic Beanstalk. And to be clear, there are certainly cases where it's okay to tightly couple that database within the Elastic Beanstalk environment. Maybe a developer is just spinning up a test environment, so tightly coupling, in that case, would be preferable for that developer, and that's fine. Think of it as sort of a disposable environment disposable database, but if you're going long term, you probably want to decouple this database from the Elastic Beanstalk environment. Let's talk about this a little bit more. RDS, and again, just to recap, RDS instances inside Elastic Beanstalk may be great for dev and test environments. Those are situations our data is probably not shared and the data is ephemeral, meaning it can just go away, especially if we're in a personal dev environment or even test. Sometimes AWS is ideal for test environments because you can spin them up so fast, tear them down just as fast, but for prod, it's a better pattern to decouple RDS from our Elastic Beanstalk environment. Benefits. We can connect multiple environments to a database, and of course, we can terminate Elastic Beanstalk without affecting that RDS database. It's loosely coupled, and it certainly helps with seamless blue/green deployments. Remember with the blue Elastic Beanstalk environment and the green Elastic Beanstalk environment, we just flip a switch and swap that URL. RDS is sitting outside of that and is not affected at all. So that's a look at Elastic Beanstalk with RDS, and just remember, the typical way you would set it up, it's tightly coupled, but it's better to have that RDS database loosely coupled. Let's talk about a few tips, and again, another recap, but it's good to ingrain these concepts. Tightly coupled database, that's fine for dev or test environment, probably not great for prod environment. Understand the why of this. The RDS database is tightly coupled to the environment. If you delete the Elastic Beanstalk environment, you're also deleting the database. If you're okay with that and you've taken steps such as backups and snapshots, that's okay too, but for long-term prod environments, it's best practice to create the database outside of the Elastic Beanstalk environment, decoupling loosely coupled. So that's a quick look at RDS with Elastic Beanstalk. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Serverless

Lambda Overview

Hey Cloud Gurus. Welcome. So, in this lesson, we're going to take a look at AWS Lambda, and we'll have three lessons on Lambda. We'll certainly answer the question, what is Lambda? And as we go through the subsequent lessons, we'll dive a little deeper into Lambda and look at it from a DevOps perspective. So let's answer the question, what is Lambda? Well, Lambda is a compute service that lets you run code without having to provision servers. You might be thinking, hey,



wait a second. That sounds like Elastic Beanstalk. But remember with Elastic Beanstalk, we can get into provisioning our servers and other resources such as databases. With Lambda, we absolutely do not touch any of that. That is all handled by AWS. We simply upload our code and we provide some metrics which we'll talk about throughout the lesson. The point that differentiates Lambda is we are only charged when our code is running, so we're not charged for servers or anything. So making that comparison again with Elastic Beanstalk, even if we don't configure the servers, we leave that up to AWS, if we have servers that are in a running state in Elastic Beanstalk, we're charged for their servers. We are not with Lambda. Alright, let's take a look at how Lambda works, but a couple of more key points. Another key point about Lambda is that Lambda is stateless, it does not hold state, so when you think about holding state, think about S3 buckets or DynamoDB or an RDS database. Lambda doesn't hold state. If you need to do that, then you need to think of a solution which includes database's DynamoDB S3. And finally, one other key point, Lambda is event-driven. So Lambda needs something to trigger the Lambda function and move into action, and we've already seen this in other parts of the course. Alright, so let's talk about how Lambda works. Alright, so we have an event source, and that is going to be what triggers our Lambda function. Yeah, I'm just going to go ahead and click on this. How is Lambda triggered? Well, numerous ways. We see S3 buckets, DynamoDB, API Gateway, we'll look at API Gateway with Lambda in an upcoming lesson. Now, the one we are familiar with already is, hey, let's upload some code to S3 and let's say the uploading of this code triggers a Lambda function, so it's similar in all of these. Basically, with DynamoDB, it could be the entry of data into a table, and it's going to trigger our Lambda function. So our event source is notified and it triggers our Lambda function, that is the essence of how Lambda works. And then we can have some actions, of course, with Lambda, whatever we decide to code in the Lambda function, so it is extremely versatile. We could have it trigger a CloudWatch event. We can move log files to CloudWatch logs. There's any number of things we can do, of course, with a Lambda function. As I mentioned, we could stream CloudWatch logs, but it all hinges upon our trigger action. Now, we made the analogy to Elastic Beanstalk, and again, like Elastic Beanstalk, we just have to upload our code, but there are some provisions we have to make for this to work properly. So what are those provisions? What are those requirements? Well, of course, we have to provide the code and it's in a deployment package. Okay. How much memory do we need to allocate for this operation? And we have to tell that to AWS. So they're not going to give us an unlimited supply of memory. We have to spec that out and provide that. How long will this run? So we provide an execution timeout option. The limit is 15 minutes. It used to be 5, it was increased to 15 minutes. As usual, we need to provide an IAM execution role, and in event source mapping, remember our event source in the previous diagram, we'll talk more about event sources upcoming. So those are our requirements. Let's focus on our execution timeout again. The limit is 15 minutes, so do we want to just say, hey, let's just allocate 15 minutes and go with that. Well remember, we are only charged when our Lambda function is running. So if we



can refine this and say, hey, we can get this done in 5 minutes, for sure why not provide that as the execution timeout? And that's something you would come by through testing your function. Alright, let's take a look at some use cases for AWS Lambda. Alright, image conversion. High-res photos can be converted to low-res photos for mobile devices. I saw this at reinvent. Alright, so we're uploading high-resolution photos to S3, very high resolution, but what if we want to see those photos on an iPhone? And of course, iPhones just get more beefy and beefy with their memory, but let's just stick with this. So we upload our high-res photo to S3. That triggers our Lambda function. So actually, this could be two Lambda functions. It could be the Lambda function which is triggered by the uploading of S3. That Lambda function could trigger another Lambda function, and remember, functions usually, you want those functions to perform one action for you. So this function is going to transcode our high-resolution photo to a lower resolution photo, and ultimately, send that lower resolution photo to our mobile phone. Lambda converts high-res photos to a mobile size photo. Now a key theme throughout the course has been automation and doing things in real time. Well, think about this. As soon as we upload, our Lambda function is triggered, and like I said, we could chain Lambda functions together, each of them having their own purpose. First one triggers the second one, second one performs that transcoding for us and sends it to the mobile phone. So this is pretty much real time. Alright, let's take a look at another example. Alright ETL, extract, transform, load. So we have an ordering system. An order is placed to our system, and let's say we're storing these orders in DynamoDB. Meanwhile, we have a Lambda function which can be triggered by any entries to our DynamoDB table. What can that do for us? Well, it can do extract, transform, load. So obviously, it's extracting that order. Again, we can chain Lambda functions together. We have a second Lambda function. It can transform this original order. Why would it do that? Well, it can format the original order into the appropriate format to be stored in Redshift for further analysis. Now, if you think about how we automate, we could just keep going with this. You know, we could have a CloudWatch event watching Redshift. Hey, the CloudWatch event could just notify someone. Hey, we have some updated data or it certainly could notify some sort of analysis program to perform an analysis on Redshift. So that's an overview look at AWS Lambda. Just remember, all we have to do is upload our code, and then we provide some metrics. How long do we want this to run? How much memory do we need? And remember, this is stateless. So if we need some sort of stateful elements here, such as DynamoDB, well, we have to include those, but Lambda itself is stateless, it will not store state for us. So with all that said, let's look at some exam tips. Alright, this is important. You see this once in a while on the exam. What is the timeout period for a Lambda function? It's 15 minutes. It used to be 5, and that can get a little bit confusing because often you'll see it in seconds. If my math is correct, 5 minutes is 300 seconds, the timeout is 15 minutes, and that's been 15 minutes for a couple years now. What if you were dealing with longer time periods? Well, there are a couple of things you can think about. First of all, and this is probably what



you would encounter on the exam. If you need functions that run longer than 15 minutes, you want to start considering step functions. Another thing you can do is run multiple Lambda functions in parallel, if possible, if it lends itself to being solved in parallel to reduce this below the 15 minutes, but also consider step functions. Alright, just know the event sources for Lambda. So Lambda can appear at the beginning of a deployment pipeline, such as code uploaded to S3, triggering our Lambda function, but it can appear anywhere it is needed in a process as long as it can be triggered. Remember how DynamoDB, entering it into DynamoDB table can trigger an ETL, which is executed by Lambda. And of course, Lambda is only limited by the skill and imagination of the function developer, so when we trigger our Lambda function, it can do anything under the sun that we can program. So that is a look at AWS Lambda, and we'll have a few more lessons on Lambda and go a bit deeper. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Lambda Versions and Aliases - Demo

Hey folks. Welcome back. So in this lesson, we're going to take a look at Lambda versions and aliases. Now let's start focusing on versions first. And when we think about Lambda Functions, what are they? Well, they're code. So we treat those as we would any other software code, think about Dev, test, prod environments, and we can have different versions of our Lambda Functions. So we want to move our Lambda Functions through that software development lifecycle, but there are specific rules in play. Aliases come into play which we'll talk about momentarily. Alright, let's talk about some rules for our Lambda Functions. So we can publish one or more version of our Lambda Function. Each Lambda function has a unique ARN. After you publish a version, it cannot be changed. And the latest version of the function is tagged as latest, so we can publish that version and then it becomes version 1. Every Lambda function starts with a default version, just remember that, and the default version is \$latest, and then we can have alias, and alias is a pointer to a specific Lambda function version. Also, like Lambda Functions, each alias has a unique ARN. Unlike Lambda Functions, the aliases can be modified, and we can update an alias to point to a different version of a Lambda Function. So in this case, our prod alias is pointing to version 1. Now, if we did some updates to our function, we created a version 2, tested it, and want to make it prod, we could simply point this prod alias to that new Lambda function version 2, and just a point here, think of Lambda alias like a pointer to a Lambda Function, and these aliases, again, can be moved around to point to specific versions of a function. So with those principles in mind, let's take a look at a couple of use cases. Alright, use case 1. We test the new version and we want limited traffic directed to it, so when we start out, we have a new version, 10% of our traffic goes to the new version, 90% goes to the existing version, and if that sounds like a canary test, that is what it is actually. Alright so incoming traffic, we



try out a canary deployment. So we're sending 10% of our traffic to version 2. Notice we have an alias test sent to this. We test it out. How's it going? Well, if testing goes well, more traffic can be directed to version 2 until it ultimately receives all of the traffic. So ultimately, remember our aliases. We're doing our test. Initially 10%, we could up this to 50. Everything looks good. Hey, let's make this our prod version. We could have a second alias, alias prod, and point it to this new version, and realistically, that alias prod would be pointing here, we just would repoint that prod alias to version 2. Let's take a look at another use case. Alright, so we have a requirement where we can have downtime, and we need rapid rollback in the case of problems with the deployment. So let's say we have Lambda Function version 1. We're testing version 2, and that alias test points to version 2. So we test this out and it tests out okay, and ultimately, we assign our alias of prod to this version 2. And of course, the test goes away. This is now prod. What if we have a problem? We get this out to prod, but we realize there's a problem. What do we do? We can quickly point our prod alias back to our old version, version 1. So there would be no downtime or very, very little downtime, negligible, let's call it that, maybe a flicker, and a very rapid rollback. Problems arise. We quickly point alias prod back to version 1. In an upcoming lesson, we're going to incorporate API gateway with Lambda, and we'll be working with aliases in that lesson, actually getting hands-on with it, so don't fear, we will definitely be getting hands-on with this. Let's take a look at some exam tips. Alright, just understand the use of versioning and aliases for various deployment strategies, blue/green, canary, devprod, so basically, we can move our aliases around and point them to the version that we want to point to, and in the case of a canary deployment, we can have multiple aliases, let's say test and prod, and we can direct a percentage of our traffic to our test environment, maybe start out at 10%. If that goes well, up to 50, if that goes well, hey, let's make it prod, we point our prod alias to that new environment. And remember, every Lambda Function has a default version, remember that \$latest. Aliases can be used for debugging our applications. So again, we can move these around, and if we run into problems, remember our rapid rollback, we can point back to an earlier version, we can point alias prod to an earlier version, and meanwhile, the version we have an issue with, we can start debugging that. When the bugs are fixed, when everything is good to go, we can move that alias prod. I'm going to call it a pointer because that's essentially what it is, point it back to the fixed version and sort of spoiled this. An alias is a pointer to a specific version number. Be careful when pointing multiple aliases to one version, so that just makes sense. In most cases, you don't want to have an alias of test and an alias of prod pointing to the same version. So that is a look at Lambda, Lambda versions, and aliases, and with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Lambda SAM Framework



Hey clock doers, welcome. So in this lesson, we're going to talk about the Lambda SAM Framework, SAM standing for serverless application model, and SAM is an open-source framework for building serverless applications, think Lambda. It provides a shorthand syntax to express functions, APIs, databases, and event source mappings, and we do this using YAML. Now that should certainly be a sort of a flashback to CloudFormation, and yes, SAM framework integrates with CloudFormation. Typically, if we want to use the SAM framework, we install the SAM CLI, and it provides a Lambda-like execution environment, and you can locally build tests and debug your applications and prepare them for integration with CloudFormation. So in this lesson, we will talk about the SAM framework. We'll see how much it integrates with CloudFormation and how we can use it just like CloudFormation. We can source control it, we can use it for disaster recovery in another region, and then we'll take a look at some SAM code, and again, see how this integrates with CloudFormation. And then finally, we'll take a look at some exam tips. So let's go ahead and look at the SAM framework. So one way to think of the SAM framework is SAM provides a bridge between Lambda and CloudFormation. Ultimately, we can store our Lambda functions and execute them within CloudFormation and within CloudFormation templates. So we have a Lambda function. Can we include Lambda code in our CloudFormation templates? Well, we've seen that already, and of course, the answer is yes. Now sort of the liaison between our Lambda function and CloudFormation is SAM, the serverless application model. So SAM gets us from Lambda to CloudFormation, and then it's simply as if we're working in CloudFormation. We have a template and CloudFormation knows how to translate this template. Recall from our CloudFormation lessons that transform function, and when we see a transform function in a CloudFormation template, we can start thinking serverless, we can start thinking about the SAM framework as being part of this CloudFormation template. Once it's a template, well, it's just the same as a regular CloudFormation template. From that template, we create a CloudFormation stack. So again, SAM is the liaison or the bridge between Lambda and CloudFormation, and let's just touch on a couple key points. Again, SAM, serverless application model, is an open-source framework for building serverless applications, Lambda. Here's what gets really cool. It is a CloudFormation extension optimized for serverless applications. We'll see this in action or in code basically, but the transform function sort of signals, hey, we're using SAM in this template, and there we go, spoiler alert. Using the transform function, we can include SAM code in our CloudFormation templates. So if we have everything in a template, what can we do with this? Well, remember we're talking about Infrastructure as Code, we want to source control our templates and that's a regular CloudFormation template or that's a SAM template that is going to execute our Lambda function. Regardless, we want to source control that. We want to maintain the integrity of those templates. Remember, the integrity of those templates is crucial because we want to have the confidence that we can reuse those templates under any circumstance, so we want to source control them. Why? Well, one reason certainly is disaster recovery. So I'm just



showing another region here. Let's assume we're in region A. I'm showing region B. Ah, hey, this is just a template. It's a CloudFormation template, it is SAM because it has a transformed function in it. Let's store that in another region in the case of a disaster and for our disaster recovery exercises. And let's hope we don't have a disaster, but we'll be ready. Alright, so we've talked about SAM, let's just look at a template and close the loop on this. So remember our CloudFormation templates, template format version, here's that by now getting pretty old date that we put in, but that's the date we put in. Remember our transform function, and this signals, hey, we're working with Lambda. We want to include some SAM in this template. Alright, so we have our transform function and this function actually, it's just performing a periodic check of a website erroring out on test failure, it's basically checking if a website is up, sort of a canary test. So remember with CloudFormation templates, the only required section is the resource section. We have to have a resource to build. Now, the caveat to that is, if this is going to be a SAM template, it also has to have a transform function because by definition, that's the only way CloudFormation would know that we're working with the server application framework here. So in this case, our resource, surprise, surprise, is a serverless function, and we're specifying a handler, we're specifying a runtime, and this may be a little bit out of date, but it doesn't really matter. This is just to show what's going on with a SAM template. Remember how we specify for Lambda the memory that we need. We specify a timeout. Now, the default timeout is 15 minutes, but again, if we don't need that, there's no use to do that. We test out our functions, and we can determine an appropriate timeout, so we're saying 10 minutes, and ultimately, it just goes out and checks a site. Now, this is a simplified template, but the key elements are in here. Our transform function, our resource, which is a serverless function, that's pretty much a SAM template. So in this lesson, we took a look at the SAM framework from an overview perspective, and it's just important to know what it is and what it does, how it integrates with CloudFormation, and certainly that transform function in the template signals that hey we're dealing with the SAM framework in this template. So with that said, let's take a look at a couple exam tips. So SAM syntax is CloudFormation syntax. It's key to realize it's YAML, but not JSON. There's no charge for SAM. You're charged only for the resources created. And remember, if we're going to use SAM, we want to set up the SAM CLI, it provides a Lambda-like execution environment so we can locally build tests and debug our applications defined by SAM templates. So that's pretty much what you need to know for the exam. So that said, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Deploying Step Functions

Hey Cloud Gurus, welcome. So in this lesson, we're going to talk about step functions with an eye on how we deploy them, and really, a step function is the implementation of a state machine, so let's first define state machines and then



we'll define step functions. In simplest terms, a state machine is a collection of states, and each state will take input from a previous state and the state will then send its output to the next state. Now these states can be called steps and that's how we get into step functions. So AWS step functions let us coordinate multiple AWS services into serverless workflows, think Lambda, and from that, we can build and update apps quickly. We can design and run workflows that stitch together services such as Lambda, Fargate, Amazon Sagemaker into feature-rich applications. Now they say a picture is worth 1000 words. I'm going to cheat a little bit here and include a bunch of words, but it's tough to sort of visualize step function sometimes and sometimes the wording helps. So remember, we're going to have multiple stages, multiple states, steps, all the same concept here.

Configure. So we define our workflow as a series of steps, such as tasks, choices, parallel executions, and timeouts. And then we populate. We connect our tasks to code hosted-in functions, instances, and on-premises servers. So when you think of code, think of Lambda functions. **Run.** We provide any needed input and run our workflow as many times as needed, and we can do this for up to 1 year with step functions. **Evolve,** so we can swap out tasks, change the order of steps, or add new steps without changing code. Now, in previous lessons, we talked about Lambda functions and the Lambda timeout of 15 minutes, and we talked about what if we exceed the 15 minutes? Well, we want to start thinking about step functions if that's the case, but we also want to think about are we doing too much with the Lambda function. Functions are best when they perform one specific function, that's good software practice. If they're doing a lot of things at the same time, that's not a well-built function, so one function for one task, but if we have multiple tasks, then we build multiple functions, and guess what, AWS step functions is going to help us coordinate the interaction between these functions that can run them in parallel, it can run one after the other when a certain state is met, it can do all of this for us, we just have to know how to use it. So on the DevOps Pro exam, when you see a scenario which has multiple steps and possibly some sequencing or some ordering or some parallel tasks that need to be run that is screaming out for step functions, so start thinking step functions when you see scenarios like that. Alright, let's take a look at a simple example, workflow, and let's first talk about some use cases for step functions. Alright, so first of all, very simply, if we have Lambda functions that extend beyond 15 minutes, we want to start thinking about step functions. Additionally, we want to start thinking about one function for one task. We might be exceeding 15 minutes simply because we're trying to do too much in a Lambda function, so we want to exercise good software practices. Step functions are good for business processes. Sometimes user intervention is necessary, so it sort of can be a mixed bag. And again, parallel execution of Lambda functions. Let's talk about a situation where in a banking scenario, and we're looking to set up a new user account. So what we could do is we could have parallel Lambda functions, two Lambda functions, one that verifies the person's name and another that verifies the person's address. Now, that's really simplified one task for one function, but we could certainly do that, and we want to keep an eye on the task states. So one



has to complete, we verify the username and we verify the user address, and basically both of these have to come back successful to move on to where we finally create the account. Now, we could have error messages in here, we could have error handling. Remember, it's just a function, it's basic software so we can catch errors, we can remediate errors, but regardless, our task states have to be successful before we move on and create the account. Now, with all that going on, again, that is too much for one Lambda function to do if we're writing good software code. That could certainly be done within 15 minutes, but those parallel tasks lend themselves to AWS step functions. And again, execution doesn't proceed until both tasks complete, and our account isn't built until both tasks complete successfully. If we have some sort of an error here, we would have another avenue to complete the step function probably having to do with throwing back an error to standard output. So in this lesson, we talked about step functions and their use cases. So for the exam know the use cases for step functions, long-running process, remember Lambda functions, the timeout is 15 minutes. So if our function exceeds 15 minutes, we want to start thinking about step functions, but for good software practice, we want to think about, hey, how much is this function doing? Is it one task for one function? That's ideal. Our step functions can involve human approval. Anytime you see something like that on the exam, start thinking about step functions. Parallel execution, as discussed in the banking scenario ideal for step functions. Business processes, again, which may involve human approval. And when you see something like a workflow, that is certainly a keyword for step functions. If you see workflow on the exam, maybe start thinking about step functions, certainly, if you're working with serverless functions. So that's a look at AWS step functions. Again, if you want to go a little bit further, check out the supplementary lesson on step functions. But with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

API Gateway Overview

Hey Cloud Gurus. Welcome. In this lesson, API Gateway Overview, we're going to take an introductory look at API Gateway, and it could be introductory or it may be a refresher. If you've gotten the AWS developer certification, you certainly would have an introduction to API Gateway. So what is API Gateway? API Gateway is an AWS service that enables you to create, publish, maintain, monitor, and secure your own REST and WebSocket APIs, and certainly between this lesson and the next lesson, we'll know a lot about REST and WebSocket APIs and the difference between the two. So we'll take an introductory look at API Gateway. We'll go a little bit deeper into how API Gateway works, and then we'll go over some exam tips. So with all that said, let's check out API Gateway. So let's take a step back further first and define an API. An API is an application programming interface and it's essentially a software intermediary, it sits in the middle here, and allows two applications or an application and a service to talk to



each other. So following the diagram, we have our API Gateway in the middle here. We have our applications, streaming dashboards, mobile apps, IoT, even on-premises applications out here in the cloud. Of course, on-prem would go through the cloud to get to API Gateway. So these apps are out in the cloud, remember API Gateway, of course, is a part of our AWS organization. So these applications establish a contract with API Gateway, and they can send communications to and through API Gateway to what we'll call our back-end AWS services, Lambda, DynamoDB, EC2, S3, any number of services. The most common use case for API Gateway is as a front end to Lambda, and we'll see that in an upcoming lesson. One way, I like to think of API Gateway is as a black box, so application developers, they can establish a contract with API Gateway and they're pretty much told what it can do for them, what the service contract is, and they don't really care what it does inside the black box as long as it does what they need it to do. Now, that general black box concept is perfectly fine for the app developers. As DevOps engineers, we of course, want to know a little bit more about API Gateway. So let's take just a little bit deeper look at API Gateway. How does API Gateway work? Again, it's like a service contract. An app gains programmatic access to AWS services through one or more APIs which are hosted in API Gateway. The app is at the front end, the integrated services and websites are located at the API's back end. So we have API Gateway and it can interact with other AWS services, complementary services such as CloudWatch, remember our automations, but its main focus is on serving these back-end services, and again, Lambda is the most common. As we know, Lambda functions need a trigger, well API Gateway, our APIs on the front end here are essentially acting as that trigger, passing information, passing requests, more specifically, to those back-end services. Okay. So we know that our applications communicate through API Gateway to our back-end services. How do they do that specifically? Using HTTP GET and POST methods. And again, just showing how our handy CloudWatch can interact with other services and be that linkage for automations from API Gateway to other services. The communication method is really the key takeaway we want from this. This is the kind of terminology we would see in a scenario question. Again, API Gateway uses HTTP GET and POST methods to access this back end through API Gateway. Two other key takeaways I want to get across in this introductory lesson, and we'll certainly go deeper in the next two lessons. There are two general types of APIs, REST APIs. REST APIs provide one-way communication. Our applications send a request through API Gateway to our back-end services. The back-end services reply with a response, and then there are WebSocket APIs which provide two-way communications. So yes, the request and response happens, but in addition to that, these back-end services can independently send communications through API Gateway to our front-end apps, and that's a general overview. In the next lesson, we'll go a lot deeper on REST and WebSocket APIs. Now security and IAM is always a part of our services, so let's talk about users. So we have our application developers working out here in the cloud, and we have our API developers working within AWS. These API developers are just, they're regular IAM users, so we need to set



them up as IAM users. These application developers, they don't need to be IAM users, they're not in our organization, so think about how we set them up, and the answer is using web identity federation techniques, so just remember those two types of users and how they would access our API Gateway service. So with that said, let's just check out some exam tips. Our API Gateway use case or use cases, the most common is API Gateway as a front end to a Lambda function, so a client makes an API request using HTTP. Lambda functions always need to trigger that request. Through API Gateway is the trigger for the Lambda function. And this is sort of an introduction to the next lesson. REST APIs use a request response model, and WebSocket APIs provide two-way communication, not just request response. The back-end can't communicate independently with the front end. So that's our overview look at API Gateway. Please join me for two more lessons on API Gateway. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

REST API vs. WebSocket API

Hey, Cloud Gurus, welcome. So in this lesson, we're going to talk about REST APIs versus WebSocket APIs. And to be a little more specific, we're going to talk about APIs which provide one-way communication versus APIs which provide two-way communication. So when I say one-way communication, I'm talking about request and response. There's a request sent and there is a response sent back, and that is one-way communication, and to describe that, the response cannot happen without the request, whereas two-way communication, our back-end channel, can send requests independently. So we're going to sort that all out. In this lesson, we will talk about APIs which use HTTP versus REST. Both of those provide one-way communication. And then we'll talk about REST APIs versus WebSocket APIs. That's really the heart of the matter. Again, one-way communication versus two-way. And then we will review some exam tips, so a quick lesson, but it's important to understand these concepts. Alright, let's first talk about our APIs which provide one-way communication, that is the HTTP and REST. So these type of APIs receive messages from a client and forward those to back-end resources. It also forwards responses from the back end to the client, but again, it's titled one-way because it is request response, the response cannot happen without the request. So what's the difference between HTTP and REST? Well, HTTP is relatively new to the scene, just introduced. I believe that re-invent 2019. What's the difference between HTTP and REST? The HTTP APIs are cheaper and faster than REST APIs. Oh boy, that seems pretty straightforward. So why not just use HTTP APIs all of the time? Well, they don't yet have all the functionality of REST APIs, and let's just talk about real quick, how does API gateway work? And we talked about this a bit before, but it's like a service contract. An app gains programmatic access to AWS services or a website on the internet through one or more APIs, and these are hosted in API Gateway. The app is at the APIs front end. Integrated AWS services and websites are located at the



APIs back end, and again important to realize this, it all starts with a request from those front-end apps, API gateway services request, sends it to the back end, back end sends a response through API gateway to the apps, and they do this using HTTP GET and POST methods to access the back-end data store. So that is HTTP and REST. Again, the most important part is one-way communication, send a request, get a response. Alright, now let's talk about REST APIs, and really, we can think of this generically one-way communication versus WebSocket, which is two-way communication. Now, the major difference between these is that the WebSocket API can send two-way communications, so think of it this way, yes, WebSocket can do request response, but with WebSocket APIs the back end can send independently messages to the front end. In other words, it's not initiated by a request, it can send its own communication without being requested anything. So if we take a look at our diagram, alright, so we have that request going from the front-end apps through API gateway to the back end. But additionally, again, that back end can send requests back to the apps independently. It doesn't have to wait on a request. So unlike a rest API which receives in response to requests, WebSocket API supports two-way communication between client apps and the back end. And again, that back end can send callback messages to our connected client, and that's really the key difference to understand. So the benefit of this two-way communication is that it supports real-time communication. Use case for this would be a real-time chat application. You're going to need that two-way communication. We would use a WebSocket API to facilitate this. So in this real-time chat scenario, our client sends a message to API gateway. And the first time API gateway sends this client to this OnConnect Lambda function, this Lambda function connects the client to the chat room. Now that the client is connected, they can send messages to the chat room. When the client is initially connected, the back end is sent a callback URL to the client. The back end can send messages using that callback URL. Now when the client is done and wants to disconnect from the chat room, API routes this message to OnDisconnect. This Lambda function disconnects the client from the chat room. Now the role DynamoDB plays in this is client identifiers, in other words, who's this client can be stored in DynamoDB. So the key takeaways from this is that the REST API and also the HTTP API support one-way communication, request response. WebSocket API supports two-way communication. The back end can independently send messages to the front end. Let's take a look at some other tips. What does REST API do? Well, we covered that. One-way communication request response. Whereas, our WebSocket API supports two-way, back end can independently send messages to their clients. Exam tips. Key use cases. Well, let's talk about the one-way format here, HTTP and REST, unless you need additional functionality provided by REST API, HTTP APIs are cheaper and faster. Do we need bidirectional traffic? Well, then we choose WebSocket APIs. First, HTTP and REST APIs. So that's a look at our APIs, one-way APIs versus two-way APIs, which is the WebSocket API. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.



API Gateway with Lambda Proxy Integration

Hey Cloud Gurus, welcome. In this lesson, I wanted to do hopefully a short lesson on API Gateway with Lambda, and that is called officially API Gateway with Lambda Proxy Integration. And additionally, I wanted to show how the versioning and aliasing can be utilized in this situation, so we'll take a look at that as well, and then we'll dive right into the demo, and we'll end the lesson with some exam tips. So we have a lot to do here, so let's get started. Alright, so API Gateway and Lambda are often paired together, and in this situation, API Gateway is basically the front end for a Lambda function, so API Gateway will accept requests from clients, applications, mobile apps, and forward those requests to our Lambda function, basically acting as a trigger for our Lambda function. Now, as you might expect, there's some integration that needs to take place and we're going to go through that using the management console. But before we do that, let's check it out graphically. So we have API Gateway and we have a Lambda function. Notice, I'm indicating a version. So the really cool thing about this is that API Gateway when it integrates with Lambda, it can reference Lambda versions and also Lambda aliases and we'll show that in the demo, so we can have a Lambda function version 1, Lambda function version 2. and version 3, maybe dev staging prod versions. There's our prod version. Well, we'll call that one test, and we'll call that one dev. So what are these? Well, API Gateway can integrate with our Lambda aliases, and remember, those aliases can move around, so wherever we're pointing prod, whichever version we're pointing prod to, that becomes our prod version, and likewise, test and dev. So the real versatility comes in, of course, with our aliases on the Lambda side, and the great fact that API Gateway can work with both Lambda versioning and aliases, and spelled out for us here, this is our prod version for as long as we want it, and if we need to change it, we can certainly do that. So that's what we're trying to accomplish here, and referring back to the title of this lesson, it is API Gateway with Lambda Proxy Integration. So it's pretty straightforward, graphically, and it's not much harder setting it up, so let's go ahead out to the management console and set up API Gateway with Lambda proxy integration. Okay, so we're going to focus on integrating an API with a back-end Lambda function, but let's just first start at Lambda, and I have this Lambda function appropriately named and a bunch of Python code, basically, it's a hello world function, fancy hello world function in a nutshell. So the idea is we want to use API Gateway to act as a proxy and send a request to this function. So let's go over to API Gateway and set that up. Alright, over in API Gateway, we want to create an API, and we want REST API, send a request, get a response, real simple. Let's click build. We'll stay with new API, name it appropriately, Lambda proxy API, regional deployed in the current region, for me, us-east-1, and let's click Create API. So within that API, we need to create a resource. So this path is already highlighted, so we're creating a resource within this path. Let's click Create resource. Here's our path. We just keep that, and I'm just going to call it good ole helloworld, keep CORS



unchecked, click Create resource. Alright, so we're doing proxy integration, and in proxy integration, the entire request is sent to the back-end Lambda function as-is, and it uses sort of a catch-all method, so let's create a method and the catch-all method is any, but let me just open that again. Notice, we have GET, we have POST, other options with the catch-all is ANY. The ANY method allows single API method set up for all of the supported HTTP methods, such as DELETE, GET, HEAD, PATCH, POST. Integration type, well, of course, we're integrating with a Lambda function, and of course, we want to turn on Lambda proxy integration, this little slider radio button. Now, we just choose our Lambda function. You just keep this default timeout as-is and click Create method. Alright, so we've created our API. Now, let's deploy and test that API, so I'll select Deploy API, select a stage, I'll just say New stage. Now we're testing, let's call it test, add a little description, and click Deploy. Now, you'll see that we have this invoke URL, and yes, that's how we're going to communicate with our back-end Lambda function. So I'm going to copy this invoke, and one caveat, our Lambda function has a couple of parameters, name and city. So to the end of this, I tack on a question mark, name=whatever, in this case, I'm just saying my name, Craig, &city equals, and I'm using Pittsburgh. So let me open up another tab, here's that invoke URL, and again, question mark name &city, let's browse. Well, what happened here? Think about that a little bit. Let's go back to API Gateway. Alright, so we have our API Lambda proxy API. Let's click on Resources helloworld. Let's go back over here. So we have our invoke URL, but we have that resource, helloworld, so this becomes /helloworld? and then those query string parameters. Let's try that again. Ah, there we go. So basically, we have confirmation that our API is acting as our proxy, sending a request to the back-end Lambda function and getting a reply, and in a nutshell, that is API Gateway with Lambda proxy integration. Click over some exam tips, and really again, two resources API Gateway and Lambda interacting, so we know we're going to have to add some permissions here, and basically, when we click that OK all of those times, that permission was being set up behind the scenes for us. You can take a look at one more related to this. How is API Gateway updated automatically? Again, it's interacting with Lambda, it needs that resource-based policy and that's why I focused on that, that little prompt of okay, do we want to give API Gateway permission? It's essential to that. How do we gain the flexibility to easily point API Gateway to different versions? We use aliases. It's really cool functionality. API Gateway is smart enough to use those versions and aliases, so that is a look at API Gateway with Lambda proxy integration. Thank you for joining me Cloud Gurus. I will see you in the next lesson.

API Gateway with Open API

Hey Cloud Gurus. Welcome to this lesson, AWS API Gateway with OpenAPI. So this is another one of those topics where we just need to know what OpenAPI is and just a little bit about how to work with it. Let's get that sorted out. What is



OpenAPI? It is a specification language for APIs that defines structure and syntax in a way that is not wedded to the programming language the API is created. Now, I certainly like analogies. Analogy I'm going to make here is with CloudFormation templates. So our templates have certain sections and most of the sections are optional, the Resources section is required, but it doesn't matter if you create a template in YAML or JSON, the rules for our template applies either way, regardless of whatever scripting language we choose, so that's sort of what OpenAPI does for us when creating APIs, and this is going to enable us to design and create our APIs in a standardized way. We go from requirements to design, configure, develop and test, publish and deploy, and again, the rules are the same regardless of the language that we choose. And conveniently to my CloudFormation template analogy, API specifications are usually written in JSON or YAML. Let's take a look at Edge-optimized API endpoints. Now, that term edge-optimized should have you thinking about CloudFront, and it's best for geographically-distributed clients and is a default endpoint type for API gateway REST APIs. So let's say we have an application, and we have a CloudFront point of presence abbreviated POP. API requests are routed to the nearest CloudFront point of presence. Ultimately, this will make its way to the edge-optimized API endpoint. So ultimately, we can use CloudFront with an edge-optimized API endpoint to serve API requests in a geographically-dispersed environment to the most advantageous endpoint, in most cases, the closest endpoint. So let's import an OpenAPI definition file into API Gateway. Let's first look at our definition file in Visual Studio Code. So at the top, openapi, every API definition must include the version of the OpenAPI specification that this definition is based on, so 3.0 isn't the most latest version, but we have to follow the rules of 3.00. Info section contains API information, title description version, and don't get that confused the 3.00 at the top that is the OpenAPI version. This second version is the version of our own API. Server section specifies the API server and base URL, and you can define one or more servers, example being maybe a production and a staging or a sandbox server, and then the Path section defines individual endpoint paths in your API. Alright, so these files could be a lot bigger than this, but I'm just keeping it simple. Let's go ahead and import this file. Alright, out at the Management console, let's create an API, and we're not going to build on the REST API. We're going to select Import. Now, we could just paste the file in here, but I'll just choose the file from my file system, and there's the OpenAPI definition. We'll just keep it regional. Notice, we could make that edge-optimized. Options to fail on warnings or ignore warnings. I tend to want to see warnings and then go from there, ideally eliminate even warnings. So let's go ahead and create our API, simple as that. And then we could start working with this doing things such as Create resource, create methods, but this is a way we can use OpenAPI to sort of standardize how we're creating our APIs. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

API Gateway Caching



Hey, Cloud Gurus. Welcome to this lesson, AWS API Gateway Caching. So our goals in this lesson are to familiarize ourselves with caching in API Gateway, and we will see similarities to caching in other services in AWS. So let's talk about how to cache an API Gateway, but before we even do that, what is API Gateway caching? Well, we've seen caching with DynamoDB in the form of DAX, and we've seen ElastiCache and it's really the same concept. We're trying to reduce our load on our back-end database, and instead of going the whole way to the database for frequently accessed data, we can cache that data using a cache in API gateway for a specified period of time, speed up our response, and reduce the load on our back-end database. So with that said, let's talk about how to cache in API Gateway. Alright, so we have API Gateway and we have DynamoDB as our back-end database, but guess what? We can create a caching cluster, and just like we've seen with some other caching technologies, we can offload some of that traffic to our database with API Gateway caching. We reduce the load on the back-end data store and can also improve our response times. Frequent query results or frequently-accessed documents can be cached for a specified time period, our TTL. Alright. Going to do a demo where we set up caching an API gateway. Let's just talk about it here for a minute. Alright, so we have created a REST API and then we start thinking about caching. Do we want caching? Well, if we do, we set the cache-enabled property. We'll see this momentarily in the Management console. How long to cache, that's that TTL property. What else would we want to set up here? Do we want to encrypt if we do? We set cache data encrypted property. One more thing we want to focus on, and as a little mini quiz when we set up a Lambda function, think about some of the properties we set up for Lambda, a key property and this all the same for an API Gateway cache. How big should the cache be? In other words, how much memory do we want to allocate for this cache cluster? And we'll see a drop-down for that. You might be a little surprised how big these caches can be. So with that said, let's wrap up by setting up caching in API gateway. Alright, now out at the Management console, I have a handful of APIs here, and I'm going to click on this Lambda proxy API, and then I'll want to click on Stages. That's where we set up caching, so we can have multiple stages. Note here, default method level caching inactive. In this case, there's only one stage. We can still set up caching. So in stage details, let's choose Edit, and here are our cache settings. We can reduce the number of calls made to our endpoint and also improve the latency of requests made to your API. Caching is charged by the hour based on cache size, that makes sense. See API pricing for details. We don't have to get into that much detail. Alright, provision API cache. Let's click here to turn this slider on, provision API caching capabilities for your stage. Key point caching is not active until you enable the method level cache, so we have to turn both of these sliders on. How much memory do we want to allocate for this? Default is .5 GB, but as you see, you can go up much higher than that. Just realize that, again, you're being charged by the hour based on this size, so some testing of this with your application calls, it's probably appropriate, and make sure you're not over-allocating capacity here because you will be charged for it. Encryption. We could



encrypt this, and we can set our TTL. It must be between 0 and 3600 seconds. Alright. With that, let's click Save, create in progress. I think this takes about 4 minutes. Alright, so we have successfully provisioned a caching cluster in API Gateway, and just remember, there's an hourly charge for that. I'll just click on edit, turn this off, and click Save, and it's deleting that cache cluster, not going to wait around for that, but that's a look at API Gateway caching. With that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

CloudFormation

CloudFormation Template Overview

Hey Cloud Gurus, welcome. So the CloudFormation template is really the heart of CloudFormation, and this is the first lesson in CloudFormation, so we'll start out of the lesson pretty much answering the question, what is CloudFormation? We'll take a look at some essentials of CloudFormation and key factors in using CloudFormation. Then we're going to start looking at our template. And remember, CloudFormation templates can be written in both YAML and JSON, so we'll take a look at that, and we'll wrap up the lesson with some exam tips. Let's start off answering that question. What is CloudFormation? CloudFormation is Infrastructure as Code, and it encourages collaboration and utilizes version control and automation. An analogy I like to make is think of object-oriented programming, and we have a class and then that instantiation of a class is an object, and it's similar in that in CloudFormation, we have a template and the instantiation of that template is our CloudFormation stack. Check out what we're talking about here. CloudFormation creates instances. Puppet puts the instances into a specific state, so remember we're going from code to instances and then we have to decide what to do with those instances, how to deploy them and configure them, so there's a lot going on. So let's check out some CloudFormation essentials and really how we use CloudFormation. So we start with a developer, and our developer wants to create code, Infrastructure as Code, using JSON or YAML. So our developer creates a template and the developer stores that template in an S3 bucket, so CloudFormation can grab that template, and we use CloudFormation, create a stack from a template, so we go from code to infrastructure. This is real infrastructure in this stack. Alright, let's look at some key factors and considerations. So we create a stack from a template and what if we want to change that stack? We can update the stack. So things we could do would be change the AMI of our instances, and that may be something that was triggered by the AMI in our region changing. We could update a CloudWatch alarm, update our autoscaling group, and to do this, we update the template, so we make the changes we need in the template, and then we update the stack with that new version of the template. And what about security? Well, we just



don't want anybody updating our stacks, so we can add some security, some policies, and they're similar to IAM and bucket policies, and essentially, we determine who can update our stacks. Now we'll talk about rollbacks and deletions in upcoming lessons, but we have to be careful with this. And certainly a main point about rollbacks and deletions is our stack fails and it attempts a rollback. It may get stuck on a single resource, and often, the troubleshooting of that is to manually delete that resource and then see if the stack will roll the whole way back. Now, talking again about updates, some considerations. How will the update affect the resource? And is the change mutable or immutable? And we'll talk a bit about change sets in an upcoming lesson where you basically get a preview of your update. Now, if we have problems, we investigate the resource that caused any issue, we can check logs, and CloudFormation provides us a lot of information during a stack build, and then the stack will be rolled back if there's a problem. Now, rollbacks can fail as well, and we'll talk about nested stacks, but oftentimes, there are dependencies in a stack and there's a certain order with which resources have to be deleted. Let's start talking about CloudFormation templates. CloudFormation template is the declaration of AWS resources that make up a stack, and it's important to note that the only required section in the template is the resource section, and it makes sense because if you don't have any resources, you really don't have a stack to create. So we declare objects, these are name pair values. So with all of that said, let's take a look at sort of a skeleton of a CloudFormation template and it will show all the elements of that template and we'll talk through those elements. Alright. So we're looking at a skeleton of a CloudFormation template, and let's go through this real quick. I'm just giving you an overview. So we have template format version. This is optional, but if you do use it, it's odd, but the version date is 2010.09.09, then we have a description, again optional, but remember with JSON, we can't have inline comments, so description may be more important with JSON than CloudFormation. Metadata can help us out with installations. Let's say that we're installing and configuring a web server. Well, we can add metadata to assist with that configuration. Parameters allow us to pass in at runtime parameters we can use. Think of like a DB name and a DB password. We can pass that in at runtime. Rules sort of go hand-in-hand with parameters and allow us to evaluate these parameters, so we're setting some rules by which we're restricting what can be entered. And then we have mappings. We'll see mappings a lot. If we're working with these two instances, a lot of times a mapping can map an instance type to a certain architecture for an instance or an AMI ID. Next, we have conditions. Good example of conditions is let's say we pass in a parameter of dev or prod, and with conditions, we can dictate the size of the servers that we config. For dev, smaller servers, prod, larger servers. Think of it like an if statement. Then we have transform and there are two kinds of these, and if you're familiar with other programming languages, think of a transform as an include, so we're including extra code into our, into our file, and the other transform is serverless and think of that as Lambda. So to add a Lambda function to our code, we would use the transform serverless. That brings us to the only required element resources, and



it makes sense creating resources is the whole point Infrastructure as Code, resources such as EC2 instances, autoscaling groups. We have to have a resource section. Finally, there are outputs. A good example would be, let's say in our template, we're creating a web server, we could output the URL to the web server. We can also output elements for cross-stack references. We'll talk about that in the section, but there's a quick look at a CloudFormation template. And again, if you need to go deeper, check out that supplementary lesson. Alright, so that's a look at CloudFormation templates, and if you're familiar with CloudFormation, this overview is probably a good refresher for you. If you're unfamiliar with CloudFormation, I would highly recommend checking out the supplementary lessons included in this lesson's materials. Let's take a look at some exam tips. We'll talk about CloudFormation templates. Again, resource is the only required section in a template. Let's talk about disaster recovery. CloudFormation templates are a good resource for disaster recovery. Basically, we have our production region, but we can store our CloudFormation templates in another region, and we can actually automate their creation, stack creation, if a disaster occurs, so they're a great tool for disaster recovery. Now with that, we want template integrity. So we want to use source control and drift detection. I have an upcoming lesson on drift detection, but basically if we have a template stored in a disaster recovery region, we want to be certain that that template remains the same as the template in our prod region. We don't want drift, and drift is the act of modifying resources outside of a CloudFormation stack. Let's say we create an EC2 instance in CloudFormation, but then we go to EC2 in the Management Console and modify that instance. Well, guess what, that instance is no longer accurately represented in our CloudFormation template and that is drift. That's a big problem. We'll cover that in an upcoming lesson. So I will end this lesson. Thank you for joining me. Cloud Gurus. I will see you in the next lesson.

CloudFormation Intrinsic Functions

Hey folks. Welcome back. In this lesson, we're going to talk about CloudFormation Intrinsic Functions. What are intrinsic functions? They are built-in CloudFormation functions which allow you to dynamically assign values to properties at runtime. So as an example, we can use an intrinsic function to retrieve the public IP address of an EC2 instance at runtime in the same stack. Key point is at runtime. In this lesson, we're going to cover seven key intrinsic functions. Their key, there are others that are important as well, some of them will be covered in other lessons, for example, the transform function covered with Lambda. So those functions are Ref, Fn:GetAtt, FindInMap, ImportValue, Join, Sub, Condition Functions, and then finally, we'll take a look at some exam tips. So we'll be looking at a lot of code, and again, for these seven key intrinsic functions, and these functions help manage our stack and they also enable template reuse and portability. Pulling in values into our stack at runtime



is very powerful and it certainly affects our ability to reuse our templates. Now, our intrinsic functions can only be used in certain parts of a stack, resource properties, outputs, metadata attributes, and update policy attributes. It can also be used to conditionally create stacks. We talked about condition functions already. Based on those conditions, what type of resources are we going to build, lean cheap resources for a dev environment or maybe beefed up, more expensive resources for prod. And really a good way to understand intrinsic functions is that term, runtime. I'll go back to the EC2 instance example. Let's say our stack is creating an EC2 instance. We need to know the IP address of that EC2 instance. Do we know it when we create the template? No, there's no way to know. Well, we could get into Elastic IP addresses, but maybe we don't want to do that, so let's just think of regular IP addresses for an EC2 instance, we create a template, it doesn't have an IP address, but at runtime, it's going to be assigned an IP address. We can retrieve that IP address during stack creation and reuse that IP address in other parts of our stack that need to know that IP address. Alright, so let's take a look at some intrinsic functions. I'm going to go out to the AWS pages and just look at the syntax, and also, just look at some quick examples. Now again, my CloudFormation Deep Dive course, if you want that deeper dive, I'm going to recommend a couple of lessons to look at on intrinsic functions, but for this lesson, let's go check out some intrinsic functions. Alright, so I'm at the page for intrinsic functions, and remember, I said seven key functions. We'll talk about some of these other ones in upcoming lessons. One of our key functions is FindInMap, and here's our syntax in JSON and syntax in YAML. And if we scroll down, here's an example of FindInMap, and again, talked about this in previous lesson. So our regions aren't guaranteed to have the same AMI IDs, so we use FindInMap to map our instance choice by region to a specific AMI ID based on architecture. So as you can see during resource creation, we're referencing that map AWS region, we just call region, AWS knows what region we're in. We call it return that using the Ref function which we'll talk about momentarily. It performs this mapping for us and gets us back the appropriate AMI ID. Here's the more compact YAML doing the same thing, much more compact, and that's certainly one of the attractions of YAML. So that is FindInMap. Let's talk about the function GetAtt, so GetAtt returns the value of an attribute from a resource in the template, remember our templates are going to have multiple resource, EC2, VPC subnets. Here's the syntax in JSON, syntax in YAML. Let's look at an example in YAML, and you can see that the GetAtt is importing values that we need in our security group, a group owner ID and a group name, and it's grabbing that from the ELB, the Elastic Load Balancer, in this template. There's the JSON version of it, different syntax of course, but it's doing the exact same thing, and this can only be done at runtime. That's the key to our intrinsic functions. Let's take a look at import value returns the value of an output exported by another stack. So this is for our cross-stack reference. So we export a value, let's call it Stack 1, and then we can import it into Stack 2 or Stack A and B if you like, and this really allows the modularization. We can separate duties in these stacks. So notice Stack A, again, is doing the export, and that is in the



output section of our template. So we're exporting, in this case, a subnet ID and also a security group ID, and realize that a subnet ID links to a VPC, so implicitly, we're attaching to a VPC, and there's how it looks in YAML. And then we can use `Fn::ImportValue` in our second stack to import the security group ID and the subnet ID. And that's going to allow us to launch an EC2 instance from Stack B into the VPC created by Stack A. Note the syntax here, a stack name parameter, and then the security group ID, so we're identifying the stack we want to pull from and then the attribute, `Fn::ImportValue`. Let's take a look at `Fn::Sub`, substitute variables in an input string with values that you specify. Basically, this is a string builder for us. You can use this function to construct commands or outputs. The key point is using items not available before runtime. So `Fn::Sub`, there's a string, and the variable name, variable value. There's our YAML look real quick, and we can use this to build URLs, that's one certain use case for it, but if you just think of it as a string builder or concatenating strings together, some of the operations you may have seen in other languages, that's what we're doing. Key point, we're doing it with values that we can only get at runtime. Notice we're utilizing the `Ref` function as well. Get a root domain name helping us to build our string. Let's take a look at `Ref`. It turns the value of the specified parameter or resource. So remember our Parameter section, we can grab that, a parameter using `Ref` or we can grab resources within our stack. There's our syntax. Let's take a look at an example. So we need an instance ID. Well, we just do `Ref my EC2 instance`, another resource further up in our template, and that's how we can grab that instance ID. `Ref` is highly used and highly valuable, so we're grabbing an instance ID basically creating an Elastic IP address for that instance. The YAML format is always a little bit simpler and more compact. And looking at our condition functions, think of them as our bullying operators with some additional, couple additional functions, `and` and `or`, if functions can help us make decisions and act upon those decisions. With an `and` or an `or`, we make the decision with an `If` function, and if conditioned, we can create resources in our template based on certain conditions. Example, I always used is a dev environment versus prod. If dev, well, then we build a smaller, less expensive server. And of course, if it's prod, well, we want the prod quality out there with, with the size and the speed, and of course, the cost will come with that, but that's prod, that's where we need it, so we can create versatile templates that would build either a dev or a prod environment for us. So in this lesson, we talked about our seven key intrinsic functions. Let's take a look at some exam tips, CloudFormation cross-stack references, and if we see that on the exam, we want to be thinking our function import value, understand how to set up a cross-stack reference, and we'll have a lesson on that actually, but most importantly, understand what function import value does for us, used to retrieve values across stacks, and that, by definition, is a cross-stack reference. So we can share values between stacks, we can split out our templates, modularize things and have our templates serve one purpose rather than a bunch all jumbled together in one template. Condition function use cases can create or not create resources based on a condition, also can be used as size resources. And again, I go back to that



dev environment versus prod, create a smaller resource for dev than you would use for a prod environment, that equates to cost savings. So that is a look at our confirmation intrinsic functions. With that, I will end this lesson. I will see you in the next lesson.

CloudFormation Wait Conditions and Creation Policies

Hey Cloud Gurus. Welcome. In this lesson, we're going to talk about CloudFormation wait conditions and creation policies, and these two are very similar. We will talk about the key differences that you just need to know and the use cases. And if you just understand a couple of different things about wait conditions and creation policies, you will be on top of the situation. So for the lesson, we'll just take an overview look at wait conditions and creation policies, and we will point out the differences, go a little bit deeper on wait conditions, and talk about WaitCondition properties and use cases, and really, they're set up very, very similar, take a look at some code for both, and then finally some exam tips. So with that said, let's talk about wait conditions. So what are we doing with wait conditions? Well, let's say we're creating a template, and of course, we create our stack, and let's say in that stack, we're creating a database instance, so our stack initiates the creation of that instance, but we need a timing mechanism. Maybe other resources need information from that database instance or it could be a case where we just need to ensure that our resource is built before stack creation completes. An example of that could be, we're creating an EC2 instance. We need to install software on that instance and it comes down to timing, we need some sort of timing mechanism, and that's where wait conditions come into play. Now, another key use case for wait conditions is let's say we're working in a hybrid environment, we're still creating a database instance, but what if that instance is on-premises? It's not even part of our AWS environment. So, we need ways to send signals back and forth, and in an upcoming lesson, we'll talk about helper scripts, but this CFN signal is used to signal, hey, our configuration of the database is done. You can go ahead and complete stack creation, but that is all dependent upon our WaitCondition, pausing stack creation until the specified resource is done. Then we can start working with CFN signal, signal back. Typically, the stack will be waiting for multiple success signals and then complete stack creation. So that's what a WaitCondition does. It basically makes the stack wait until a key resource is created, and a great example would be an on-premises database that's interacting with our AWS VPC. So again, what are wait conditions? They pause the execution of stack creation and wait for a number of success signals. Also, this is over a specified time period. There's a timeout period, so if we specify a timeout and don't receive those success signals in that specified period of time, then our stack creation is going to fail, and we can go into our stack and troubleshoot it. It will probably jump out at us in an error message that the timeout period expired. That's the easy part. Sometimes the difficult part is why did the



timeout period expire? What happened? Why didn't we receive the success signals? So WaitCondition properties. Count, that's the number of success signals that CloudFormation must receive for continues the stack creation process, so remember, it's waiting. It's waiting on our hybrid database creation to complete. Now, I'm just using that example. There are any number of examples. It's waiting on those success signals. Handle. That's just a reference to the WaitConditionHandle used to signal the WaitCondition. There is our timeout. The length of time in seconds, and if my math is correct, let's say 5 minutes, that would be 300 seconds to wait for the number of signals that the count property specifies. So what are some use cases? It's a timing mechanism, and we're coordinating stack resource creation with configuration actions that are external to the stack creation. Think of a hybrid scenario, on-premises resources such as a database, and we wait on the provisioning of resources, more specific waiting for that on-prem database to be configured. We could be waiting for NAT instance, and this is a great example of the timing, so that NAT instance has to be created and configured before our private instances can attempt to access the internet. Alright. So that's wait conditions. What about creation policies? Creation policies are very, very similar. There is some overlap. A creation policy pauses the creation of a resource until a requisite number of success signals are received. Sounds just like their WaitCondition, right? Here are the differences. There are two key differences. First of all, so a WaitCondition is itself a CloudFormation resource. Just like an EC2 instance or a VPC subnet, it is a resource. A creation policy is an attribute associated with resources. So an EC2 instance, for example, could have an attribute of a creation policy. Wait conditions is a resource itself. Those are the two key differences, that's why I'm not spending a lot of time even going over creation policies because it would be repetitive. They're set up pretty much the same other than that difference one's a resource wait conditions, one is an attribute. Now, if you want to go deeper, I'll have a couple of supplementary lessons referenced in this lesson from my CloudFormation Deep Dive course. So we know what wait conditions are and creation policies as well. We know their use cases and the key differences. Let's just take a look at some code now. Alright, let's start with a WaitCondition, and again, notice that it is a CloudFormation resource. Notice also, for EC2 and autoscaling, we recommend you use a creation policy, instead of wait conditions. It's a recommendation, my personal experience with wait conditions and EC2, it's a little bit sketchy. I've had better results with creation policies for EC2 as their recommendation suggests. Here's our syntax. Just remember what the count is for number of success signals. We're waiting on our handle and our timeout. How long should we wait? And just remember, the handle is used to signal this WaitCondition. Let's check out an example. WaitCondition waits for the desired number of instances in a WebServerGroup. So AWS recommends using creation policies for EC2 and autoscaling, but yet, the example for wait conditions is an AutoScalingGroup. From personal experience, I've had better results using creation policies with autoscaling groups. Notice we have a WaitHandle in here. That's also a resource type, but here is our WaitCondition. It



depends on `WebServerGroup`. Here's our handle. Notice `WaitHandle`. Here's our handle timeout, that is 5 minutes and our count `WebServerCapacity`. So it is waiting for actually the desired capacity. So success signals, let's say desired capacity is three. It's waiting for three success signals. And again, remember stack creation is paused. Once we go into the `WaitCondition`, the stack creation is paused. It's waiting on the success signals. If it gets the success signals, stack creation resumes, if it doesn't within 5 minutes, our stack creation fails, and then we start troubleshooting. Now let's look at creation policies and notice the difference already. `CreationPolicy` attribute. It is an attribute of a resource. Here is our syntax, and this would be for an `AutoScalingGroup`, but it will make a little more sense. Let's look at an example. So we have an `AutoScalingGroup` that is a resource, `CloudFormation` resource. Notice `CreationPolicy`, again, it is an attribute. It is part of this `AutoScalingGroup`, so it doesn't need a handle because the `AutoScalingGroup` knows where it's at. It's part of the `AutoScalingGroup`, but it does have a count and it has a timeout. Just note that this syntax is different, PT 15 minutes, so it's setting a 15-minute timeout. Other than that, wait conditions and creation policies operate the same way. So again, certainly remember the use cases, creation policies are recommended for EC2 and autoscaling groups. Over wait conditions, wait conditions are great for hybrid environments, so we're waiting on the creation of good example and on-premises database. Alright, let's take a look at some exam tips, and on the DevOps pro exam, probably need to know, hey, what's the difference between these two, wait conditions and resource. Creation policy is an attribute of a resource. Wait conditions are good for hybrid environments. Creation policies are good for EC2 and autoscaling groups. Wait conditions can be used though, and I probably just gave away some of the answers here for EC2 and autoscaling. What's recommended? Creation policies. Which one is an attribute of a resource? Creation policy. Which one is itself a resource? Wait condition. Other than that, just remember what could go wrong. Well, something went wrong with the creation of our instances. There are error messages that get sent back, sometimes cryptic, sometimes it's pretty obvious, and then within the stack as well, we'll see error messages. So that's a look at wait conditions and creation policies. And again, if you want to go further, I'm providing some supplementary lessons from my `CloudFormation Deep Dive`, and with that, I'll end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

CloudFormation Helper Scripts

Hey, Cloud Gurus. In this lesson, we're going to take a look at `CloudFormation` helper scripts. So we'll just describe what are helper scripts, and then we'll start looking at the individual helper scripts which include `cfn-init`, `cfn-signal`, we've already seen that a little bit in the last lesson, `cfn-hup`, and `cfn-get metadata`, and remember, our intrinsic function regarding metadata, so this is going to interact with our `CloudFormation` metadata. We'll get into details and then some exam



tips. So let's start talking about helper scripts. Alright, let's first define what are helper scripts. They're actually a set of Python scripts to install software and start services on EC2 instances, think of bootstrapping and bootstrapping our EC2 instances during startup, so these helper scripts are called directly from our CloudFormation template, they also work with resource metadata defined in the template. And what are use cases? Well, we've seen one already. We want to send signals back to our stack, and we want to configure in Bootstrap our instances or update our instances. So let's take a look here and just get our mind around this graphically. So we're in CloudFormation, we have a CloudFormation template, and we are building a stack, and we have an autoscaling group. Let's say we're building the group, and we have our helper scripts to help us out. Again. They are cfn-init so it retrieves and interprets resource metadata to install packages, create files, and start services. So let's talk about a LAMP stack real quick which has a database and its PHP, and most likely, it's a MySQL database, the A is an Apache server, so we can install packages and we can start our web server service, all of this from cfn-init. We'll take a look at the code momentarily. Like other lessons in this CloudFormation section, I'm going to reference my CloudFormation Deep Dive course, and again, as the name implies, it goes probably deeper than you need for the exam, but it's certainly good information if it interests you and if you need to sort of ingrain the concepts better. Cfn-signal, we've seen that already, and we're signaling between our wait condition and our stack. What's going on with that wait condition? Do we have success? Can the stack complete? And remember, the stack pauses during a wait condition, so it's waiting for those signals, and it better get those signals before the timeout expires or the stack creation fails. Alright, let's talk about cfn-help. It is a daemon. It checks for updates to metadata and executes custom hooks when changes are detected, so it's a daemon. It remains running on our EC2 instance after the instance is spun up, so this is actually for updates, and a good analogy I'd make for this one is think of how CloudWatch events watches an S3 bucket for updates to code. This is similar. Our instance has been created and already bootstrapped, but there may be updates, and this is a way to perform those updates if we have the daemon running, and cfn-get metadata wrapper script that can retrieve either all metadata for a resource or path to a specific key or subtree of the resource metadata, this sometimes can be used for troubleshooting, so we're retrieving that data seeing if everything's straight with this data and trying to identify a problem. Let's take a look at these in the code example. Again, and it is a helper script, and it reads template metadata and things it does is so it retrieves the data, but then it can install packages, write files to disk, enable and start services. So let's take a LAMP stack, for instance, so that LAMP stack has a database, we can install packages for that database, and we can write files to disk. What about LAMP stacks? The web server, Apache web server, we can enable and start and stop our web server. That's the function of cfn-init. Here's a little bit of the syntax. Here is an example of cfn-init in actual code. So it's getting the stack name. Alright, what stack are we operating on here? What resource are we installing to? And then config sets are just



instructions. Okay, so we're going to install and then run. So I'll just stick with LAMP stack. We install the database, we install Apache, and then run, we would start up the web server. Let's take a look at cfn-signal. We already have a good idea of what's going on here working with wait conditions and creation policies. And remember, when we go into a wait condition or creation policy, we're waiting on success signals. So if we have an autoscaling group with a preferred size of three instances, we'd better receive those success signals, three of them within the timeout period or the stack creation will fail. And check out the supplementary lesson included in the lesson materials if you want to go deeper on these. I go pretty deep on all of them and analyze some code. Sometimes if you're new to CloudFormation, that can get confusing looking at a big jumble of code, but it's certainly there if you want to go that route. Let's check out cfn-hup. Again, it's a daemon, it's running on our instance, waiting for updates to metadata, so that's a way we can update our instances. Notice one thing. By default, it runs as a daemon. We would have to specify node daemon to get it to run once and then exit. And finally, get-metadata, and it does just what the name implies, retrieves metadata, print out the standard output. Why do we want to print it out? We could get into a troubleshooting situation, and that's why we would print it out just to review our metadata, make sure everything's straight and make sure there's not an issue within the metadata. That's about what you need to know for helper scripts for the purposes of the exam. Check out the supplementary lessons if you want to go deeper. Let's talk about some exam tips. So the helper scripts are written in Python, and they're used to install software and start services on EC2 instances that you create as part of a CloudFormation stack. Remember the metadata section of our template, these scripts work with that metadata that's defined in our template. And remember, cfn-signal, it's used the same way with wait conditions and creation policies, and think about an autoscaling group, your preferred size, let's say it's 3, so you're waiting for success signals from 3 instances in that autoscaling group and it has to come within the timeout period that we specify, otherwise, stack creation fails, and then we start troubleshooting. So that's a look at CloudFormation helper scripts. We have a bit more to discuss on CloudFormation, so let's move on to another lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

CloudFormation Stack Protection

Hello Cloud Gurus. Welcome. So we talk about CloudFormation as Infrastructure as Code, and the code we're talking about is going to follow the same software development lifecycles that we have with other software projects. We'll have dev and prod and test environments. Well, when we get to our prod environment, how do we protect our CloudFormation stacks? So in this lesson, we'll talk about just that. So we'll take a look at a diagram and just outline what we're trying to do when we protect our stacks and how we protect our stacks, and then we'll



get into the details and look at some policies, and we'll also go out to the AWS Management Console and check out how we protect our stacks, and then we'll just go over some exam tips. So a short, but very important lesson on protecting our CloudFormation stacks. Alright, let's take a look at CloudFormation stack protection. So obviously, we start with a CloudFormation template and we build our stack. Now again, if this is a production environment, we need to implement some protection for this stack. We want the proper personnel working with these stacks, such as deletions and updates, and we don't want just anybody being able to delete our production stacks. Now to get it a bit more complicated, let's say we have multiple production stacks and multiple architects with admin access to those stacks. So how do we protect our stacks from accidental deletion or updates? Well, there are four ways. We can implement termination protection on the stack, and we'll see that momentarily. We do that. We can do that from the Management Console. We can also do it from the CLI. We can add policies to our stacks, directly to our stacks. We can also add resource-level policies to resources within our template. We'll take a look at that as well, and always remember, our first line of defense is simply IAM, so we can add protection to our CloudFormation stacks through IAM, and in general, we could have groups in IAM, groups of admins, those admins can delete stacks, and then we have groups of regular users, those regular users cannot delete stacks. Now that's kind of simplified, but that gets the idea across. So the termination protection is the most direct way, but always remember, IAM is a crucial part of our security for all services in AWS. Alright, let's go a little bit deeper on these four ways of protecting our stacks. So I'll just work clockwise. Termination protection. We can add termination protection to our stacks, and in particular, our production stacks. So we enable when creating the stack, and we'll take a look at that momentarily. The lead actions against the stack will be denied. It's pretty simple. Nested stacks flows, the termination protection flows from the parent stack. Again, IAM policies. Just like all the other AWS services, it's a good starting point for our security. Deletion policies used to preserve or back up a resource when the stack is deleted. Now, remember we're talking about a resource, so this deletion policy is going to be in our template, and we'll take a look at one of those also momentarily. And then finally, stack-level policies. So we can add policies directly to our stack. Upon stack creation, all update actions are allowed on all resources. That's the default, but we can add policies which restrict the update actions that we want to restrict, so let's just take a quick look at a couple of these again from the Management Console, and also, we'll look at a template, and it'll just drive home the concept and aid in our retention of these concepts. Alright, so we're looking at a CloudFormation template in JSON for an S3 bucket. Very straightforward. We're just creating an S3 bucket, public read access, and it's going to be a website. There's our index.html, there's our error.html. Now, here's the thing. We want to retain our S3 bucket if our stack gets deleted, so we have a deletion policy of retain, and that's basically how we can protect our S3 bucket from being deleted. Now, what if we don't care if that bucket is deleted? Well, let's take a look at another conveniently named



S3noretain. Whereas the deletion policy, well, we don't care. This bucket can be deleted if the stack is deleted. No deletion policy. So it's key to understand that the default is to not retain. So to retain, we want to implicitly add this. Now, remember with S3 buckets, here's the catch. For an S3 bucket to be deleted, what do we need to do? That bucket needs to be empty, and that's a key point because if the bucket's not empty, that will keep the bucket from being deleted if we want it deleted. Alright, so I'm out at the Management Console in CloudFormation. Let's go ahead and create a stack, and I just want to show this real quick. I'm just going to use a sample template, a LAMP stack, not even going to create this stack, but I want to show just in case you haven't seen it before our stack protection. Alright, so for the LAMP stack, I went ahead and populated all the passwords, DBUser, InstanceType, KeyPair. Next, advanced options. First of all, here's where we can apply a policy to a stack, and it's simply an AWS policy in JSON, and with that, we can add protection to our stack, and then furthermore, stack creation options. Let's expand this. Ah, there we go. We can enable termination protection. So there's all our lines of defense for protecting our stacks. At the very root IAM policies, we can add stack protection, termination protection, stack policies, and then remember, within the template itself, resource-level protection. So in this lesson, we took a look at the four ways we can protect our CloudFormation stacks. Just know these four ways and how we go about protecting our stacks, and let's just take a look at some exam tips. Why is stack protection so important? Well, it's Infrastructure as Code, and those production stacks are just like our software applications. We certainly don't want our stacks deleted in production. And remember, our DR purposes as well, confirmation is a big part of that, and we want to protect the integrity of our stacks, and we certainly can get into drift situations as well, which is not good for DR. Who can delete stacks? Again, we went over this. We can start with IAM and have groups of admins and maybe some lead developers, and in those groups, give them the proper permissions to delete stacks, and just remember, some of our chains of automation and notification, other services AWS Config, CloudWatch, and SNS can be used to signal when our deletions occur. So that's a look at protecting our CloudFormation stacks. And with that, I'll end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Updating CloudFormation Stacks

Hey Cloud Gurus. Welcome. In this lesson, updating CloudFormation Stacks, we're going to talk about the ways we can update stacks and some of the consequences we have to consider when updating stacks. So we'll take a look at the ways we can update stacks, and maybe most importantly, update behaviors the way our resources behave when we perform an update on them. And finally, some exam tips. So this is some foundational knowledge that we should have. I haven't seen a lot on updating stacks on the exam, but if you want to take a deeper dive, I'm providing two supplementary lessons, one on



direct stack updates and one on change sets. So let's take a look at updating stacks. Alright, so we're in CloudFormation, and we have a template and we have created a stack. So do we need to change a stack parameters or resources? I guess probably the most common example would be we created a web application. We chose an instance size that is turning out to be too small, so we want to up the instance size. Now we could just delete the stack, but we could also update the stack. Now, if the size of the instance is a parameter, we could update the parameters when we do an update or we could change the resource of the EC2 instance, we could change the size of that instance. So when we perform an update, CloudFormation compares the changes and only updates the changed resources. So there are two ways to do this. We can do this with a direct update. We just go into our CloudFormation stack and click Update, and we can also do it with a change set. And basically this change set is another template we can apply to our stack, sort of overlay it on the existing stack and make changes. Now, the benefit of the change set is that it sort of gives us a preview of how these changes are going to affect our stack, sort of like a hey, what is the domino effect of making these changes? And what if we don't like the domino effect? Well, no harm done. We can back out of the change, maybe redesign or make changes based on what we've learned and try the update again. The key takeaways from this diagram, two ways to make changes to our stacks, direct updates and change sets. The change set gives you an opportunity to preview changes and make a no-go decision on if you want to make those changes. Now, what could be affecting whether we want to make those changes? Well, it's the behavior of the resources, the parameters within those resources that could affect our decision, so let's talk about change behaviors. So when we make changes to the resources in our stack, let's say we are applying a change set, and again, we have the opportunity to preview those changes. What are we looking at specifically? We're looking at the update behaviors of the resources in our stack, and some these behaviors are update with no interruptions, so it updates the resource without disrupting the operation of that resource and without changing the resource's physical ID. Update with some interruption, updates with some interruption and retains the physical ID. Now, let's think about this a minute. So what if our company has downtime policies and really frowns upon unscheduled downtime? Well, we're doing an update with some interruption. We're going to want to schedule downtime, so that's really where these update behaviors become important, update behavior of replacement. The resource is recreated and given a new physical ID, and that's certainly behavior that we want to be aware of as well. How do we know the update behavior of our resources? Well, there's really only one way, and AWS has a document called the AWS resources and properties type references. I suppose if you work in CloudFormation constantly, you may memorize some of these update behaviors or maybe all of them, but it'd be a pretty tall task. So if you work in CloudFormation occasionally, it's good to know this document exists and have it handy. Let's take a quick look at this document. Alright, out at the AWS resource and property types reference, information for all AWS resources, so there's a lot of



information here. Again, certainly not something that we want to waste time memorizing, but we want to know it exists, and it's a handy tool in our toolbox. Let's go to EC2, and specifically, EC2 instance. Now, our instance has a bunch of properties. Notice, this is where we find our update behaviors. No interruption. Let me click on that. Confirmation updates the resource without disrupting operation of that resource and without changing the resources physical ID, and then there's some interruption, the instance might have some interruption. What CloudFormation in EC2 reconfigure the instance replacement, recreates the resource during an update, which also generates a new physical ID. Now, not only do we have to be aware of this physical ID and it possibly changing and where are we referencing this at? For instance, think about it, an autoscaling launch template could have some IDs in the template. So that's one consideration. And of course, the other consideration is okay, do we have downtime, a little bit of downtime? Is that allowable, or are we in a no-downtime situation? So we have to understand these update behaviors and plan our updates accordingly. So let's go back, and again, just looking at EC2. Let's look at some key properties availability zone. If that changes, that requires a replacement. IAM instance profile. Well, we can change that with no interruption. Image ID. AMI ID probably could have guessed that, hey, if we're changing an image, an image ID, that update behavior is going to be replacement, so that's a look at the update behaviors for our services, and specifically, EC2. We must be aware of these when we're working with CloudFormation and performing updates. Alright, so that's a quick look at updating CloudFormation stacks. It's not a huge topic for the exam, but certainly something we need to know again. I have two supplementary lessons where I go much deeper on direct updates and change sets if you have that interest and want to learn about that. So with that said, let's check out some exam tips. Should I use direct updates or change sets? Well, it depends probably on what environment we're in, but if you're in a production environment, why not go with change sets? It's not like it takes a bunch more time. They provide a preview of the impact of the change and also a final go/no-go option. And if we don't like what we see in that preview, we can back out, rework the change set, and try again. Direct updates are more acceptable in dev or test environments, but they don't provide that final impact assessment. How do I utilize update behaviors? Well, it's going to dictate when to modify resources. If we're sensitive to downtime, we need to be aware of our update behaviors when we're making a change. Example be, do we need to retain the physical ID of a resource? And is this going to be a full replacement? Again, just be aware of these update behaviors. Key takeaways from this lesson. Two types of ways we can update, direct updates or change sets. Remember our update behaviors and also where to find these update behaviors, that document AWS resource and properties type references. I'll include a link to that in the lesson materials. So with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.



CloudFormation Nested Stacks and Cross-Stack References

Hey Cloud Gurus. Welcome. So in this lesson, we're going to talk about CloudFormation nested stacks and cross-stack references. Now, to explain all of this, let's think about what CloudFormation is. Well, it's Infrastructure as Code with the key word being code. So we want to utilize software best practices when creating our templates. We want to modularize our code and make it reusable, and the nested stacks and cross stack references give us opportunities to do this and make it more manageable. So let's first talk about nested stacks. So we have a root stack, and we find ourselves in a big organization. We're creating templates and we notice that these templates, we keep creating the same resources, such as an S3 bucket or an autoscaling group, and one of our team members has a strong software background and he starts thinking about code reuse and asks the question, hey, can we break this S3 bucket out into its own template and reference that template throughout our other templates? And the answer to that is yes, and that's where nested stacks come into play. So we can have a root stack and we can have multiple children stacks. Now I used S3 as an example, but what if we happen to be creating the same or a very similar database over and over again? Or what about we're creating the same autoscaling group throughout our organization? We can pull these resources out into their own template. I'll use autoscaling group. We have one template for our autoscaling group, and then we can reuse that. That modularizes our code. Guess what? It's easier to maintain one template, we know where the template is, rather than having this resource of an autoscaling group throughout many templates in our organization. Ultimately, we're creating, in this case, network resources, but its origin is in code, and we can use software best practices. So how do we create a nested stack? Well, we create the nested stack within another stack, and when I say that, the stack is the instantiation of a template, we create a stack resource within a template. So in this case, the root stack would have a stack resource referencing a child's stack, and there can be multiple levels of this. It gets a little bit messy if you go too many levels, I'd I'd keep it pretty conservative with the number of levels, but it's really just one line of code to create a nested stack, and of course, pulling the resource out and creating it in its own template. It's pretty darn easy. So what do we want to remember about this? Hey, CloudFormation is infrastructure as code. So let's go ahead and take a look at a nested stack in a template and it'll tie this together a bit. So I'll use the example, hey, we want to reuse an S3 bucket. We have separated that out into a template. What do we do? And it is this simple. We need another resource. That resource is an AWS CloudFormation Stack. Where's this stack coming from? Well, where's the template to the S3 bucket? In this case, we're providing a URL to an S3 bucket, and we can reuse that, and it's not an exaggeration, might reuse it 100 times hundreds of times. If we ever need to make a change to this template, well, guess what? We only need to change it in one place, so that's the benefits of a nested stack. Now, just a little bit of extra



information in our output section. If we want to reference that nested stack, well, it's just a resource like any other resource in our template. So we reference it myStack, well, hey, that's the name of it, myStack, and it would reference our S3 bucket. It's no more complicated than that for nested stacks. Alright, so we have a good handle on nested stacks. Let's talk about cross-stack references. Now again, this helps us with sort of modularizing our code, but there's additional advantages as well. So what is the problem here? Well, again, it might come down to just having a template, one template that just keeps growing and growing and sort of getting difficult to manage. We have multiple teams trying to access this template and work in it at the same time. An example I'm going to talk about a network team and an application team. Common software term is spaghetti code. It's just one big blob of code and difficult to manage, but our network team and our application team are always colliding working in the same template, sort of fighting over the same template. Why don't we give them their own template? So yes, they need to work together. The network team builds its resources in a template. The application team maybe needs to reference something in that template. Well, beforehand, it was in the same template, no problem, but can we have two templates and reference values between them? Basically pass values to another template? And as you might expect, the answer is yes. So we have a network stack, and I'm saying stack, but again, that is the instantiation of our template. We split these apart. It was one template, we split the network stuff out into a network template and the application into an application template, and it really comes down to a line of code in each template. So in the output section of the network template, let's say that the application team needs a VPCID and a subnet ID from the network. Well, the network in the output section of their template just performs an export, a VPCID and a subnet ID. That export is available to this application stack. It becomes another line of code in the application side and it's an import. So the application team needs to create an EC2 instance. Well, what do they need? They need, of course, to place that EC2 instance into a VPC, so what do we need? We need VPCID and a subnet ID. Well, we have it available to us because of this export command, and that's how we can clean up this messy situation of two teams or even more than two teams sort of battling over the same template. Hey, let's modularize things a little bit. Let's share information, and this is how we would do it. Now just to tie this all together, let's look at these templates. I'll pull up a Code Editor and we'll see how simple this is. It's just a couple lines of code. Alright, let's take a look at cross-stack references. I'll stick with the example. We have a network team, we have an application team, we gave them their own templates and split them out, but our application team is creating an EC2 instance. Well, what do you need with that? We need to launch that into a VPC. That VPC is owned by the network team, so there has to be some communication going on here. So we implement that with a cross-stack reference. So looking at our template, this is just a garden variety template creating a VPC and all of the elements within that VPC, but our application team needs some information about our stack. They're going to create an EC2 instance. So, how do we send



them information? It is really this simple. This export, that is the key to creating a cross-stack reference. I'll call this the upstream stack. We create an export, and we're sending the StackName VPCID. Likewise SubnetID, same way, StackName SubnetID, we're exporting it security group. So by exporting it, we're making it available to other stacks. Now, how does the application stack get that information? Alright, so they are creating an EC2 instance for which to run their application on. Of course, they need to launch that instance into a VPC. Remember how VPCs work. Our SubnetID is tied to a VPC, so we don't even need that VPCID. We do need the SubnetID. This is how we get it, `Fn::ImportValue`, and we reference that stack SubnetID. Remember how we're exporting StackName SubnetID. We want the security group for this EC2 instance. Same situation. Again, implicitly, we're getting the VPC by having this subnet, subnets tied to the VPC. It's that simple. Export, the value that's needed or values. Downstream `Fn::Import` the value or values that we want. And having that information, well it's necessary because of course the EC2 instance needs to launch into a VPC, so it is that simple with cross-stack references. Alright, so that is a look at cross-stack references. So just to recap, we have nested stacks, and we also covered cross-stack references. There's some overlap in what they can do for us from a software perspective. They certainly can help us modularize our code. More specifically, the nested stack can help us reuse the creation of resources such as S3 bucket autoscaling group. If we're doing that over and over again in a template, why not pull that out? Make it its own template and reference that. So that helps modularize things, but it also helps in maintenance. Hey, if we need to change that, we only need to change it in one place. The cross-stack reference, again, we have this unwieldy template, we can split it out into multiple templates and pass information from one template to the other. Talked about code reuse modularization, and hey, we're going to source control our code as well. Alright, so just some questions just to drive these concepts home. Hey, we need to manage stacks separately. What are we going to do here? We can use cross-stack references. In our example, we broke it out into a network stack and an application stack, passed information from one to the other using a cross-stack reference. Need to manage stacks together? Well, we can still modular, but we can use nested stacks and manage it through a root stack. Need to modularize our code? Well, really they both do that, but if you had to choose one for modularizing code, think of nested stacks, think of the power of having an S3 bucket in one template, and this is not an exaggeration, maybe reusing it 100 times. And of course, want to share data between stacks? Well, cross-stack reference. Alright, just a few more points to sort of drive home these concepts. Want to reuse blocks of code that is used often? Example database, autoscaling, load balancers, certainly should know this by now. That is a job for a nested stack, break those resources out into their own template and we can reference them in a root template. Alright, next one. Huge template shared by multiple groups in your organization. Why don't we create smaller group-specific templates, and that's a job for cross-stack references. Do we need a subnet ID network stack? We know that, cross-stack reference. So with



that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

CloudFormation Drift Detection and Remediation

Hey Cloud Gurus. Welcome in this lesson, CloudFormation Drift Detection and Remediation, we're going to talk about CloudFormation Drift, and if we get drift, how we can fix it. Now, I haven't seen a lot of coverage of drift on the exam, but it contributes to our overall knowledge of CloudFormation, and therefore, important, and there are other important concepts that could keep us away from drift situations, such as CloudFormation stack protection, protecting our stacks and who can change those stacks, and in a more general sense, the concept of least privilege. So if we have EC2 instances created in a CloudFormation stack, we want to protect those instances in the stack, but also, in the EC2 management console, we want to dictate who can change those and limit really who can change those. Detection may not be heavily covered on the exam, but it certainly touches concepts that are heavily covered. We'll define what is drift. We'll play a little drift game, and we'll talk about fixing drifts. and then we'll talk about some exam tips. So let's go ahead and talk about drift detection in CloudFormation. In a previous lesson on cross-stack references, we had two stacks, a network stack and an application stack, and we shared information between those stacks, and one of the resources in the stack was an EC2 instance. So just to use a scenario that's fresh in our mind, let's see how we can introduce drift into this stack. Not that it's a good thing, but it does happen. Alright, so what is drift, and why is it a problem? Well, drift is a problem when we change resources that were created in a CloudFormation stack when we make changes to those resources outside of the stack. Why is that a problem? Well, we then have a CloudFormation template which created that stack which doesn't accurately represent the current topology of that stack. Let's say we changed the size of an instance outside of the stack. Well, we basically invalidated that template. It no longer represents the stack. Is that a problem? Well, let's say we're using that template for disaster recovery purposes. We're expecting to see a certain stack when we use that template. Another problem can occur when we do updates to a stack the stack is different, the update may not work when we delete a stack. If something has changed in that stack, it may cause the deletion of the stack to fail, several things that could go wrong when we introduce drift into our stacks, either accidentally or just out of not caring. So let's keep this scenario in mind, and we will focus on the instance in this stack. Let's take a closer look. So our developer goes into the EC2 console, not the CloudFormation stack, and adds an Elastic IP address to the EC2 instance, and we need an IP address that doesn't change. So the EIP is a good thing, right? Well, it's a good thing if it would have been changed in the CloudFormation stack by doing an update to the stack. A couple months later during a disaster recovery drill, the drill fails because the original template does



not have the expected EIP. Why doesn't it have it? Well, it wasn't updated properly. The EIP was created in the EC2 console which has nothing to do with our stack. So if we wanted an EIP address in this stack, we should have performed an update, either direct update or change set, made sure the template was changed to reflect this new EIP address, updated the stack, and then when we needed the template months later in a disaster drill or worse yet in an actual disaster, the template would reflect what we wanted it to reflect having that EIP address. So one part of prevention is just knowing, hey, do not alter stack resources outside of the stack, but the other part of prevention is our stack protection techniques and our least privileged techniques. Alright, now that we know a little bit more about drift, let's play a game here. Coming soon to the game show network, drift or no drift. You inform your supervisor that he's changing a database connection string in the RDS console. You're pointing that out tactfully hopefully. Is this drift? Yes, it is. It's in the RDS console. Now, the assumption we're going with here, of course, is that these resources were created in a CloudFormation stack. Months after stack is deployed, you perform a stack update to add an EIP address to an EC2 instance. Is this drift? No, that's actually how the above scenario should have been executed. Stack update. Stack updates do not create drift. We make a change to a security group's ingress in the EC2 console. Is this drift? Absolutely. It's an EC2, it's not in the stack where the EC2 instance was created. We change an EC2 instance size in the management console. We, later, update the CloudFormation template to reflect this change. Is this drift? Huh. Sort of a trick question here. We change the EC2 instance size in the Management console outside of the stack. Technically, that is drift. Then we catch ourselves. We later update the template to reflect this change. We basically remediated the drift. So was this best practice? No, it says we later update. How much later? Who knows? Five minutes later? Okay. I guess that's alright. Two days later, no good. It's best to update stack resources in CloudFormation. There was drift for a short time. However long this later happened to be. Alright, let's talk about fixing drift, and best way to avoid drift is to avoid drift. Let's use principles of least privilege, stack protection, as well as just good practices of updating our stack resources in the stack. Detection steps, well, we can use the management console to run drift detection on the entire stack or individual resources. We can also use AWS Config, and there's a rule, stack drift detection which can detect drift. Now, even though we may have introduced drift, we may have wanted the change. So we have to evaluate the details and decide to revert or keep changes, but either way, the proper way to remediate drift would be to update the stack accordingly. Let's say that an EC2 instance was changed in the EC2 management console. Well, like the scenario above, we would want to change the template to reflect that change if we want to keep it and then perform a stack update. Ideally, we focus on the techniques to avoid drift and we never have to remediate. For the exam, I think this is the proper level of coverage for drift. Like I said, I haven't seen a lot of drift on the DevOps pro exam, but if you want to take a deeper dive on providing two supplementary lessons, one on drift detection and another on drift remediation goes a lot



deeper, probably more deeper than we need to go for the exam, but certainly, if you have an interest, check out those lessons. So that let's us check out some exam tips. Drift can relate closely to stack protection. Again, protect our stacks, avoiding drift. We use policies and principles of least privilege. What's the big deal about drift? Well, it's a big deal if your template does not reflect the stack that you want to create, especially in a disaster recovery scenario. You hopefully catch that in a drill if you have drift rather than an actual disaster, and it can also cause failures in stack updates and/or stack deletion, so it can be pretty damaging. Tools to automate drift detection, sure, we can use AWS config to detect drift and then some of our automations link it up with SNS and and notify us. Bottom line, limit who can change stack resources within the stack and also outside of the stack and make those changes using CloudFormation updates to avoid drift altogether. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

CloudFormation Custom Resources

Hey folks. Welcome back. In this lesson, we're going to talk about CloudFormation custom resources. So we'll look at an overview of custom resources and actually how they operate via a diagram, and we'll check out some code and see how they're set up in a CloudFormation template, and that will really cover what we need to know for CloudFormation resources, and we'll check out some exam tips. So let's go ahead and dive into custom resources. So why do we need custom resources? Well, first of all, AWS is always adding new resources and sometimes CloudFormation lags in having those resources within the CloudFormation code set. Other things are, we may need to do special circumstances that are not covered in any kind of process we have in AWS. So what if there are tasks we want to complete that are not supported? Well, that's a case for custom resources. So for the most part, when we think about custom resources in CloudFormation, we're using Lambda functions, and we know the power of Lambda functions. So think about it, we have a limitation. There's something in CloudFormation that we can't do. Well, we can introduce Lambda. Lambda can pretty much do whatever you can code it to do. So let's see how CloudFormation custom resources work. So we have a CloudFormation template and we store it in S3, and we start CloudFormation and we want to build a stack. We initiate the stack building process. CloudFormation retrieves that template from S3. Oh, wait a second. There is a Lambda function in our CloudFormation template, so CloudFormation deploys that Lambda function. The Lambda function executes, does whatever it's designed to do. It could do any of thousands of things. We'll take a look at a specific example shortly. The Lambda function runs and then it's going to return its payload to CloudFormation, whatever information CloudFormation needs, and then stack creation proceeds and the stack creation completes, and here's our stack resources. I just showing an autoscaling group EC2 instances, it could be anything. So let's say we're



creating an EC2 instance. What could we possibly need from a Lambda function? Now, this example is a bit outdated, but let's say that we want a specific AMI to build that EC2 instance with. The Lambda function can go look up an AMI ID for a certain region, return that to CloudFormation and create the instance with that AMI ID. Again, there are probably better ways to do that now. Another example would be a password checker. Sometimes we create a resource in CloudFormation and I'll use a database as an example, you enter the database password, but there's no verify password box. What happens if you get that password wrong. So with Lambda function, we could provide a password verification before we create the database and the password. Alright, so let's check out a template anatomy for this, and I like to have a little checklist when I'm working with Lambda functions. Alright. These are the steps that take place for custom resource in CloudFormation. First of all, CloudFormation retrieves your package source from S3. Remember, it can be in S3 or it can be inline. CloudFormation then deploys the Lambda function. Lambda runs and returns data to CloudFormation, and then CloudFormation moves forward in its template and deploys other resources. And here is the little checklist. I'm showing it in a fancy graphical format, but it's basically a checklist. CloudFormation templates require three elements to utilize Lambda functions, a custom resource, which has one property, a service token, we'll look at that in some code here momentarily, and remember our template anatomy. Remember that custom resource section. So we need a custom resource. We need a Lambda execution role. We've heard that before. Execution roles. So basically permissions for a Lambda function and permissions for any resources we're operating on, and of course, we need the Lambda function. Now, here's the thing about Lambda functions. We can have these functions in line in our CloudFormation template or a zip file in S3. My personal rule of thumb is if the Lambda function is fairly small, I don't know, maybe under 50 lines, I keep it in line in the template, but if it's larger than that, that can get a bit messy for other folks looking at the template, then I'll zip it up and keep it in S3. So if we remember this little checklist, these functions are pretty easy to implement, so let's take a look at some code and just talk through this real quick. It's fairly straightforward. It's more straightforward than say our CloudFormation helper scripts. Alright, so let's look at a JSON template, AMILookup.json based on an instance type selection shown here in our parameters. We have a custom resource Lambda function which will retrieve the appropriate AMI for the instance type and the region that we're in and that with that AMI, we can build the EC2 instance. Now there are other ways to do this, but that's not as important as just understanding how these Lambda functions work and I always like this little function because it's pretty darn straightforward. So we have our parameters instance type, module name, name of the JavaScript file AMILookup, S3 bucket, the name of the bucket that contains your packaged resource S3 key name of the zip package. Remember, this is very similar to working with S3 in the Developer Tools suite, code build code deploy, so we name our bucket and then we name the key is the name of our zipped file. Alright, let's scroll down. We have some mappings



here for EC2 instances. We're creating an EC2 instance. Here we go AMIInfo, custom resource named AMIInfo, so this could actually be AWS confirmation, custom resource ::AMIInfo or this Custom::AMIInfo. So remember our checklist. We need to define a service token. Our custom resource has one required property and it is the service token, and that service token specifies where CloudFormation sends requests to. Where is it sending requests AMIInfo function. Ah, lo and behold, AMIInfo function is our Lambda function. So this is our custom resource. It has a service token, it's referencing our Lambda function. Our Lambda function has a handler. Remember our execution role. That's necessary. Remember our little checklist. Now, our handler is referring to module name. Where have we heard that before? We've heard that in parameters, name of the JavaScript file AMILookup. So we have the Lambda function defined in our code, but the Lambda function, there's no code in here to do what we need it to do to get an AMI ID based on instance type and region, so that means that our function is in an S3 bucket. What's the name of this thing? Module name again, it's up here. We specify the runtime, and we're using Node.js and the timeout. Remember our Lambda execution role? Here is the actual execution role, giving Lambda permission to act on our behalf. Notice some of the permissions Effect Allow, CreateLogGroup, CreateLogStream, Describelmages. So went a little deep on the code here, but the key things to remember are, hey, we want to work with Lambda custom resources. We need to create the custom resource, that resource needs a service token which is calling our Lambda function. In this case, our code is not inline, so that means it is in an S3 bucket. Remember, we need our Lambda execution role. Here, we're referencing it. Here's the actual role. And within our Lambda function, we have a handler, it's going up, getting that module runtime and a timeout. So it can get complicated looking at this code and the code is bouncing around, but I just work with that checklist, custom resource Lambda execution role, Lambda function. Within that function, we have a handler runtime, a role, and a timeout. That is it. In this lesson, we took a look at automation custom resources, and I think Lambda for the most part when talking about custom resources. Let's look at a few exam tips. Why do we need them? Well, confirmation doesn't support every resource in AWS or there might be tasks that are not supported. We can use a custom resource, and we can code anything under the sun with Lambda and just remember how they're created. Custom resources are implemented using Lambda functions. Do the following. Retrieve our package from S3, confirmation deploys the Lambda function. Lambda runs and returns data to CloudFormation, and then stack creation continues and ultimately completes. So that's a look at CloudFormation custom resources. So I'm going to end this lesson. Thank you for joining me Cloud Gurus. I'll see you in the next lesson.

CloudFormation StackSets

Hey Cloud Gurus. Welcome to this lesson, CloudFormation StackSets. So of



course, we covered CloudFormation in the configuration management and Infrastructure as Code domain. Now we're in resilient cloud solutions. Why? Well to sum up CloudFormation StackSets, we can use one template to deploy stacks, and not only one template, but one operation, one overall operation. We deploy stacks in multiple regions and multiple accounts, one operation, that's sort of StackSets in a nutshell. So in this lesson, we'll take an overview, look at StackSets, take an important look at StackSet accounts, and look at permission models for StackSets and just see how the StackSets would fit into a pipeline, just one example, and we'll wrap up with some exam tips, so let's start off by learning a little bit more about StackSets. So with StackSets, we can create, update, or delete stacks across multiple accounts and regions with a single operation. Now granted, this takes a little bit more time than deploying in a single account, single region, one stack certainly takes more time, and this could become very large. We could have several regions and many accounts, but also very powerful. Now, it's important to understand that we're going to have an admin account, multiple accounts. One of these accounts has to be the admin for our CloudFormation deployments. So we have an admin account, but then we also have target accounts, multiple accounts in multiple regions, and just for size considerations, this is fairly small, but again, this could be five, six, seven regions, as many regions as support CloudFormation StackSets, and many accounts. So from the admin account, we're deploying the stack and then it gets deployed to all of these regions. Now it may not be necessary to get into great detail, but we can configure how we want to deploy. For instance, we can set up maximum concurrent accounts, how many deployments we do at one time, we can do it by percentage, and then what if something goes wrong? We can set up the fault tolerance that we'll accept before rolling back. Alright, we talked a bit about StackSet accounts, the all important admin account. Again, it is the account in which we create the StackSets. So when we need to manage our StackSets, this is the account that we log into. Target account is the account in which you create, update, or delete one or more stacks in your StackSets, so we can get rid of one of these stacks. We don't have to do them all at once, although you certainly can do that. What do we need to work between accounts, of course, a trust relationship between the administrator and the target. And with that said, let's talk about permission models. So we can use two permission models. Self-manage permissions, so we can deploy StackSets to any AWS account where we have permissions. In that case, we create the IAM roles that set up the trust between the accounts, and there has to be that trust. Now, here's a recurring theme. The other type of permission model is service-managed permissions. We can deploy stack instances to accounts, here's a key, managed by organizations. So sometimes up front, it almost seems like a pain to set up AWS organizations, but it helps with so many other things such as CloudFormation StackSets and AWS control tower and landing zones. Once you get organizations set up, it's really, really useful with other services. Another key point about service-managed permissions is that if you go this route, if you have organizations configured, the IAM roles and trust are created for you. So that's



very helpful if you're going to be using StackSets on a recurring basis. Alright, and let's just sort of visualize StackSets in a deployment pipeline environment, so we're working with StackSets and our dev team pushes changes, and of course, that can be a trigger, changes to the repository triggers our pipeline. CodeBuild deploys the changes in the build spec file which includes the changes against the StackSets, so this certainly can be a deployment pipeline. Within that pipeline for our StackSet, our admin account can then deploy those changes to our target accounts, sort of an oversimplified end part here. Again, could be many accounts in multiple regions. Alright, so that's an overview of CloudFormation StackSets. Let's take a look at a few exam tips, might not be heavily covered on an exam, but certainly want to know about StackSets. Certainly, the basics. Where can you deploy CloudFormation StackSets? Well, you can deploy to multiple accounts and multiple regions, and this can become very large and naturally as it grows almost exponentially, this can take longer, considerably longer than just deploying one stack and one account. Which types of accounts are required to use StackSets? Well, we need an admin account first and foremost, and of course, you need targets. You need target accounts, and this could be many accounts, again, across multiple regions. What are the two permission models used to deploy StackSets, either self-managed or service-managed, key points. Self-manage, you have to create your IAM roles, service-managed organizations, AWS organizations is assisting us, and will create those roles for us, so that's a look at confirmation StackSets and how they contribute to resilient cloud solutions in multi-account, multi-region environments. So with that, I win this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

Configuration Management

Working with ECS

Hey Cloud Gurus, welcome. So in this lesson, we're going to work with Elastic Container Service, otherwise known as ECS. So what is ECS? It's a highly-scalable, high-performant container management service that supports Docker containers and allows you to easily run applications on a managed cluster of EC2 instances. Now, a good way to think of this coming from maybe a software background is ECS is a wrapper for Docker. Underneath ECS is going to be Docker. So in this lesson, we'll take a look at Elastic Container Registry, which is closely associated with ECS, and we'll see how, and then we will take a look at in-depth of how we work with ECS, and we'll also do this from the Management Console. We'll work with ECS a little bit in the Management Console, and then we'll review some exam tips. So with all that said, let's take a look at Elastic Container Service. Alright, so before we talk about ECS, let's talk about ECR, Elastic Container Registry, and maybe a good analogy to make here is the



container registry houses Docker images, ECS deploys those images. ECR is a fully-managed Container Registry, it makes it easy for developers to share and deploy container images, Docker images, and ECR integrates with, as you might expect, ECS, also EKS, Elastic Kubernetes Service, also another container service, and AWS Lambda. So let's first take a look at this graphically. Alright, so we have Docker and essentially Docker Hub. So with Docker, we can write code and package it as a Docker image. Now, we need a place to store these images and that's where ECR comes into play, or we store them in Docker Hub. So we have our Docker image, we want to store that, and in ECR, we can compress, encrypt, and control access to these images. We can version tag and manage image lifecycles all in ECR. What comes next? Well, you can probably guess. ECS comes next. ECS is actually deploying those images again. ECR houses the images, ECS deploys those images, or EKS, Elastic Kubernetes Service. So we run these containers, and we pull the images, and we can run these containers anywhere. Remember, that's the hallmark of Docker, so we can run these anywhere. Everything we need is contained within our Docker image. So Docker Hub is a service provided by Docker for finding and sharing container images with your team. Think of ECR as an AWS version of Docker Hub, and another key point is it is a private container registry. For the exam, just make sure that you understand this diagram and the relationships here. Docker container images, we can store in Docker Hub, certainly work with those, but we want to be AWS centric, and this is a managed service. We use ECR. ECS can work with ECR, as well as EKS. Now let's take a deeper look into ECS, both from the diagram and also the Management Console. Alright, so ultimately, we're going to have an ECS cluster. So we have an ECS cluster and it contains these tasks and ENIs, Elastic Network Interfaces. So we start with a container image, and we use a Docker file to create a container image. Alright, where does ECR come into play? We publish the image to a container registry. Now remember Docker Hub, but this is AWS specific we're using ECR. Remember, ECR is a private registry. A task definition, and this is step three, specifies parameters for the application, including which containers to run. So once we create a task definition, then we can go ahead and deploy. Now, our container image is pulled from the registry and the task definition pretty much contains the instructions on how to deploy this to our ECS cluster. One thing I want to mention about this is that in our cluster, we can use either EC2 or Fargate, and Fargate is a serverless technology, and we can use it with ECS to run containers, and there's actually another upcoming lesson on Fargate, so I don't need to go real deep on that. We'll also see Fargate in this lesson when we do the demonstration in the Management Console, but just remember that we can use Fargate serverless or EC2, and of course, with EC2, we'll have to manage those servers. Alright, now, this is sort of new to some of us and probably a situation where let's do it from the Management Console, and it'll tie these concepts together. Alright, out at the Management Console, we're going to deploy a container on Fargate, and of course, Fargate is serverless, so we don't have to do anything with servers. We're going to create a cluster. We'll need to create a task definition and then we'll have to create a service. And ultimately,



we'll have a little web app deployed in our container on Fargate. So let's click Create a cluster, give our cluster a name and here are our infrastructure options. Let's just stick with Fargate, and let's create. This won't take long. Alright, so our cluster has been created. Then we need to create a task definition. Think of this as a blueprint for our application. Tell us which Docker image to use for containers, how many containers to use in the task, and the resource allocation for each container. So let's click Task definitions. And we want to create new task definition and new task definition with JSON, and it provides sort of a skeleton for us to get started, but I have another AWS-provided task definition which I'll provide in the lesson materials. I'll paste that in. Notice, of course, Fargate, it's using a public Docker image. So that's our container running TCP on port 80. Notice CPU size and memory, and there's a command. Basically, this is a test app, so there's some HTML which ultimately ends up being our index.html. So let's click Create. So we have a task definition. We have a cluster. Can't do anything just yet. We need a service. Let's go back to clusters. Let's click our cluster, and on the Services tab, let's click Create. Alright deployment configuration. How is our app deployed? Let's click on this drop-down. Here is our task definition which is telling it how and what to deploy. Just give our service a name, desired number of tasks to launch, we'll keep it at one. Now down here under Networking, alright, we have a VPC. We have some subnets, use an existing security group. There's the default. We certainly could create a new one. Notice public IP is turned on, after all, it's a website. Let's click Create. Alright, ECS deployment in progress, it takes a few minutes, and surprise, surprise. A lot of what goes on under the hood in AWS happens in CloudFormation. Let's go over there, and we have a stack. So it's creating the ECS service. Go to Resources. You see a service creation in progress. We can keep an eye on it here, or ultimately, we'll know up here. Alright, our service has been deployed successfully. One of one tasks running. So well, first of all, let's go over to CloudFormation, and yes, as expected, that completed. So by now, we should have our container deployed on Fargate and our application deployed within the container. How do we check that? Let's go into our service. All looks well. Note our CPU and our memory that we specified in our task definition. Let's click on our task, the Task tab, and then let's choose our task. I'll just close that. Alright, configuration, public IP address, open address. Let's click that. And there's our sample web app. Now of course, certainly be deploying a more robust application, but this gives us the idea. Create a cluster, create a task definition, create the service, and we have our Docker container deployed to Fargate, and our application is up and running. Back to the diagram. We used ECS to deploy a Docker image, and we had a task definition. Remember, the ENIs, so dynamically, we can assign an IP address, and we use that IP address to browse to our application. So with all that said, let's take a look at some exam tips. Alright, little quiz action here. What uses Docker images in task definition to launch containers as part of tasks? Well, that's pretty easy, ECS. What's used to create Docker images? Docker file. What uses Docker images to launch containers? Task definition. Docker Hub is a public container registry. What is a



private ECR? What are pulled from the container registry? Ah, five for five. So in this lesson, we took a look at Elastic Container Service, and it also requires us to understand Elastic Container Registry, as well as a little bit of Fargate, and we will have an upcoming lesson on Fargate. So with that said, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Fargate Overview

Hey Cloud Gurus, welcome. In this lesson, Fargate Overview, we'll do just that. We'll take an overview look at Fargate. It's not heavily covered on the DevOps pro exam, but it's maybe becoming more of a focus, and we certainly want to know what it does and how we use it. We'll start with an overview look, then we'll go into a demo. I'm going to keep the theory a little bit light. Fargate is pretty darn basic. Let's get eyes on Fargate and how to configure it, then we'll take a look at some test cases, and finally, some exam tips. So with that said, let's get into our Fargate overview. So with Fargate, sort of a pretty standard setup here, we're going to have an application load balancer fronting Fargate, and we're going to use elastic network interfaces. So we have these Fargate tasks and basically, here's the key with Fargate. Fargate is dealing with containers. If there's two things you want to remember about Fargate, first of all, Fargate is serverless, and maybe even more importantly, Fargate is dealing with containers. So when we think about containers in AWS, we think about ECS, and yes, Fargate works with ECS, and underneath ECS is Docker. So those are the key takeaways for Fargate. And another thing to take away, okay, what is a Fargate task? The Fargate task is a logical group of one or more Docker containers that are deployed with specified settings. And what is Fargate? It is a serverless container service. Let's talk about a few more attributes of Fargate before we get into the demo. Fargate is like EC2, but instead of giving you a virtual machine, you get a container. It's a compute engine that allows you to use containers as a fundamental compute primitive rather than EC2 instances, and it's serverless, so we don't have to manage the underlying instances. So all we do is build our container image and then we set some specs, CPU, memory requirements, find your networking and IAM policies, and then just launch. Alright, let's just tie some of these concepts together with a demo. Okay. I'm out at the Management Console and I've gone into CloudShell, and we're going to do a lot of work from the CLI. And what we're going to do is create a Fargate cluster, register a task definition, a Linux task definition to be specific, and run that task and maybe a couple other scenarios. Alright, let's first create a Fargate cluster, and the command is `AWS ECS`, and that's just our context, `create-cluster cluster-name` and we'll just call it `fargate-cluster`. I'll go ahead and run that, always a really good sign when you see some JSON like that, no error messages. There's a cluster ARN, `clusterName`, status `active`, on and on. Alright, now before we can run a task on our cluster, we need to register a task definition, and the task definitions are lists of containers grouped together. Over in IAM, we need a task execution role. I'll go



to Roles and I'll just search on ECS, and what we're looking for is this `ecsTaskExecutionRole`, and in that is the Amazon policy `ECSTaskExecutionRolePolicy`. Now, what we're going to need is the ARN for this role. So I'll copy that. Alright, what we're looking at here is a task definition in my Code Editor. Here's the task ARN we just copied from our role, and the Docker image, port mappings, FARGATE, cpu, and memory, and we want to upload this file. Let's go back over to CloudShell. From CloudShell, we can do Actions, Upload file, very nice and convenient here. There's our task definition file, and in just a few seconds, it uploads. So I'll do a `ls`, and all the way over here is our task definition. Our command is `register-task-definition` where input file is `taskdef.json`. Let's run that. My syntax was a bit off. Let's try this one. Alright, looking much better. Let me just scroll up here real quick. There's the start of our JSON task definition, Space bar, just paging through and there's the end. Alright, so we have successfully created and registered a task definition. Sort of trust, but verify, let's do a `list-task-definitions` command, and the one we created is `sample-fargate`. I have a few others out here, and that's just fine. So we've registered a task definition for our account. We can now create a service for that task. Now, our task requires a route to the internet and it could be a NAT Gateway, Elastic IP address, private subnet, but let's just do a public subnet. So what we're going to need is a public subnet and a security group. So let's go over to VPC. Alright, so I have a public subnet, copy that subnet ID, the subnet is in this VPC. Let's jump over there, and a security group within that VPC, so I grab that security group. That all needs plugged into my command. Let's go back over to CloudShell. Alright, here's the big long command, `aws ecs create-service`, clusters are `fargate-cluster` service-name `fargate-service`. Here's our task definition, `zycount1`, launch type `Fargate`. Here's our network information. The subnet security group assigned public IP enabled. We're giving a public IP address to our task. Let's go ahead and run that. Any time I see pages of JSON, I become very optimistic. No error messages, here's our task definition. I'll hit Space, created, time, platform `Linux`, there's our subnet security group, IP address, service role, I'll hit space. Alright, so we have created a task definition. We've created a service for our task definition. And once again, trust but verify, let's list our services, and we just have one, and these are just some common commands. Let's describe services, see what that does for us. Deployment configuration, employment circuit breaker, a rollback, not turned on, not much, but it provides an ID for our service. Just maybe a different take on the same information. Little bit more information here. Our service has reached a steady state, deployment completed, one task started, no failures. Let's try one more command, list tasks, run that, and it throws us back that task ARN for our task. Now, before we wrap up, let's bounce over to ECS in the Management Console. There's our `fargate-cluster`. With one task running, all looks well. Let's go back to CloudShell, and to clean this up, we delete the service, `aws ecs delete-service`, name of the cluster, name of the service, and we're forcing a delete, Space bar, I'll hit Space one more time, and we had no error messages. Good news. After we delete the service, we can delete the cluster. Delete the service



first and our command is `aws ecs delete cluster` and the name of the cluster. Let's run that. Notice already the status is active, so this isn't going to happen instantaneously, but we can be sure that our service and cluster will be deleted. So let's look at working with fargate and ECS from the command-line interface. Alright, let's recap what we did. Well, first of all, we created an ECS cluster and Fargate remembers working with containers in AWS, that is ECS, underneath of it is Docker. So we have this Fargate task, and with that, we created a task definition to tell ECS how do we want this deployed, memory and CPU and all of that good stuff. We also created this fronting application load balancer. It's going through an elastic network interface and routing traffic to our Fargate task, our containers. And just as an aside, we can always attach CloudWatch logs and log log information from our containers into CloudWatch. Couple other key points, just like EC2 instances, these containers can be added in public or private subnets, and of course, in a private subnet, if we want internet access, we have to go through those methods such as adding NAT Gateways. So a public load balancer accepts traffic from the internet, routes it to the container through the Fargate tasks ENI. Now, if we had a private internal load balancer, it would only accept traffic from other containers in the cluster in this area. So I want to give that visual in the Management Console just to tie these concepts together, realizing that maybe you haven't worked with Fargate yet. Key takeaways, again, it is a serverless service and it deals with containers, think ECS underneath the VCS, think Docker. So let's take a look at some exam tips. Why use Fargate? Well, if you don't want to manage servers, sort of like Elastic Beanstalk, you just want to deal with your applications. And also if you want to work with containers, a serverless Compute Engine for containers, it works with ECS, and also, Kubernetes. How do I focus on applications? Use Fargate or Elastic Beanstalk. They're similar, but again, not the same. Fargate is focused exclusively on containers, and I'd probably answer this question. Suited for application, applications it's not server focused, strictly focused on containers. There's also a similarity to Lambda. Well, Fargate is serverless, but again, containers, Fargate, there's that keyword connection, use cases, microservice architecture applications, batch processing, machine learning, and it's really good for migrating from on-prem to the cloud. ECS and EKS can both use Fargate to scale, load balance, and optimize container availability. So that is a look at AWS Fargate, a serverless service focused on containers. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

AWS Cloud Development Kit (CDK)

Welcome to this lesson, AWS Cloud Development Kit, otherwise known as the CDK. So in this lesson, we'll define what is the CDK, why we're going to use it, how it works, and then we'll do a demo mostly about configuring the CDK and then build a small stack just to get a flavor of the CDK, and we'll finish up with



some exam tips. So let's go ahead and get started with what is the AWS CDK. What is the Cloud Development Kit? A framework developed by AWS that allows you to define and provision cloud infrastructure in code, and you might be thinking, hang on here, whoa, timeout. That sounds like CloudFormation. Well, good point. The CDK uses CloudFormation, think of it as a wrapper on top of CloudFormation. Another expression, CloudFormation is under the hood of CDK, and when we do the demo, we'll go out to the Management Console and see that we've built a CloudFormation stack. So the CDK is using high-level languages like Java, C#, but again, under the hood, it resolves to a CloudFormation template, and from that template, the stack is built. So why use the CDK? Let's take a look at the benefits of the CDK over CloudFormation. So let's say we need to configure networking resources, everything to get internet access to and from our instances, VPC subnets, route table, security group, bash and hosts, internet gateways, NAT Gateways. Well, as you're probably aware, that could take 500 to 1000 lines of code in CloudFormation depending. YAML is a little bit more compact than JSON, but that could be done roughly with 50 lines of code in the CDK using these more expressive and powerful languages. So again, remember CloudFormation, of course, is using JSON and YAML scripts. CDK is using higher-level languages, Java, C#, certainly higher level Go, Python, TypeScript, JavaScript, so we're really performing the same function as we are with CloudFormation templates. This is more geared toward developers who are more familiar with those higher level languages. But just remember, ultimately, it does resolve to a CloudFormation template. And with that said, CDK can fit into pipelines and automations just like we would with CloudFormation. So in this scenario, developers where maybe previously or maybe they still do develop CloudFormation templates and we apply that to our pipeline. We have our automation when they commit that code. Well, we can do the same thing with the CDK. Now let's talk a little bit about some details of how that works. The CDK uses three major building blocks to provision infrastructure. There's the app, there is the stack, and then there are constructs, so let's break that down. So the app is the root of our constructs tree. It consolidates all stacks and constructs in one application that can then be deployed on AWS, almost think of it like a folder. Stack is a single unit that holds the AWS resources in the form of constructs and can be used for development. Now, when we get down to the constructs, well, we're seeing these icons for AWS resources. A building block that can contain a single AWS resource in here is a very important point or multiple AWS resources combined. So in other words, we could have a construct, think of just the classic configuration, EC2 instances in an autoscaling group behind a load balancer. We could have a construct that builds that for us. So in CloudFormation stacks, we do it one resource at a time. This is grouping constructs, and these constructs are stored in construct libraries and we can reuse that code, that's a real key to the CDK. So for the exam, really know what the CDK is, its relationship with CloudFormation and this idea of constructs and construct libraries, these building blocks and reusing those building blocks to build our infrastructure. With that all said, let's



take a look at the CDK from the AWS command line. Alright, now let's work with the CDK. Now remembering our diagram, we had apps inside of apps, we had stacks and inside our constructs. So we're going to create the app and then we're going to add code to the app, and the code will create resources within the stacks. Alright, so we're creating our app. Note, the `mkdir` command, then we change it to that directory. Now we're going to initialize our app, and just note the language, TypeScript. Alright, let's run a manual build of our code. I'm going to open another tab and let's go over to CloudFormation. But first from the command line, let's list the stacks in our app. Alright, so we built a demo CDK stack. Alright, so out at CloudFormation, we see that our stack is being built, and this is really an empty stack, but we built this stack using the CDK, then we can start adding resources to it, auto scaling groups, load balancers, S3 buckets. Now this isn't a programming course, and I didn't want to get too deep into this, but this is a good stopping point. We see how we can build CloudFormation stacks using the CDK with much less lines of code in languages that are geared towards developers, Java, C#, I use TypeScript. So let's take a look at some exam tips for the CDK. What benefits the CDK offer over CloudFormation, write less code, get more done, may be less lines of code by an order of magnitude of 10, roughly. What is the relationship between CDK and CloudFormation? Well, as we saw, the CDK code is transformed to CloudFormation. Ultimately, we end up with a CloudFormation stack. So just like CloudFormation, what automations can CDK participate in? It can be integrated in pipelines just like CloudFormation. So when we commit our code, we can have an automation set up to start our pipeline just as we would with cloudformation templates. This is important. What contains reusable constructs, including custom constructs, well, that AWS construct library, so we can reuse that code, and again, it's more than one resource, there can be multiple resources together. So that is a look at the AWS CDK. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

AWS AppConfig

Hey Cloud Gurus, welcome to this lesson on AWS AppConfig. So what is AppConfig? Well, it's part of Systems Manager, and we use it to create, manage, and quickly deploy application configurations. So in this lesson, we'll take an overview look at AWS AppConfig, and we'll answer a few questions and talk about the differences between AppConfig and CodeDeploy, talk about some use cases and benefits of AppConfig, and this is another server where it's very helpful just to see it being set up and see the components of it, so we'll do a demo and set up AppConfig, and we'll finish off with some exam tips. So with all that said, let's take an overview look at AWS AppConfig. So with AppConfig, we can reduce errors in configuration changes to our applications. We can control deployment of these configuration changes across our applications, and we can control changes quickly across a set of targets. Now, what are those targets? Well, it's where we're holding our apps, so we're talking about containers and Lambda with Lambda



Functions, and of course, EC2, and we can also deploy to IoT and mobile apps. Alright, so what is AppConfig because it does really sound a lot like CodeDeploy. And what is the difference between AppConfig and CodeDeploy? CodeDeploy helps us roll out new versions of our applications. Now, of course, first time we roll it out, we use CodeDeploy, but when we roll out new versions of that application, we also use CodeDeploy. Well, what about AppConfig. AppConfig helps apply configuration changes to running applications, and those configuration changes are values in code that we might need to change fairly frequently, and we want an easy way to do that. Examples would be feature flags where we can just flip a switch and turn on features, maybe A/B testing, debugging and logging attributes, think about environment variables for dev, prod, testing, QA, operational tuning like throttling limits or connection limits, or maybe a database failover configuration. So CodeDeploy deploys our application, AppConfig deploys configuration changes to those applications, that is a key difference. Now let's take a look at a pipeline, sort of clarify things. So our developers commit their code, and I'm just showing a typical pipeline, CodeCommit, CodeBuild, CodeDeploy, and then we deploy our application to our target. In this case, an EC2 instance. Where does AppConfig fit in? Well, it sort of circumvents our main pipeline, but we can deploy our configuration changes to our target, again, where our application resides and configuration changes to that application, so that's a very key difference and really helps in understanding what AppConfig does. So with that understanding, let's talk about some use cases for AppConfig and some benefits. Alright, so use cases introduce changes to our apps and tests with production traffic. So again, we're not deploying a whole application. We're deploying configuration changes quickly. Turn on new features during a product launch or announcement, and we'll have feature flags. We'll see how you set that up in the demo. We can allow premium subscribers to access paid content. So it would be like an allow deny based on if they are a premium subscriber. And we can reduce stress on our apps. When a dependency impacts the system, AppConfig sort of has a much smaller footprint than doing a full blown new version deployment. So we have a process here, we have a pipeline, we reduce errors, and our configuration changes, we can deploy changes across targets quickly, we'll see how to set that up. We can update our apps without interruption. An example of that would be just a feature flag turning on a feature on our prod server. We can control the deployment of changes, this is another one that will make a lot more sense when we do the demo momentarily, and we can roll back changes if errors occur using CloudWatch alarms. So we can tie our AppConfig deployments to specific CloudWatch alarms and roll back if our alarm goes into alarm state. With all that said, let's go to the demo and check out how we set up AppConfig and this will tie all of this information together. Alright, before going over to AppConfig, real quickly, just wanted to show something I did beforehand, which is set up this role. Systems Manager CloudWatchAlarmDiscoveryRole. So this establishes the relationship between AppConfig and CloudWatch if we take a look at the trust relationship created



here. So there's a trust between our CloudWatch alarms and AppConfig, and the policy, I have the policy up here again, establishing that relationship between AppConfig and CloudWatch. So again, if something goes wrong in our configuration deployment, we can trigger a rollback. With that said prerequisite set up, let's go over to AppConfig. Alright, and we're going to click Get Started. So we're going to create an application, and think of an application as an organizational construct like a folder, and within that folder, we're going to have our configuration data that we want to deploy. Then we can just give our application a name and an optional description. We also have optional extensions and tags, and this is really for our notification automations, but let's just keep it simple. We'll create the application. Now, the next thing we want to do is create an environment, so I'll click that, and an environment is a logical deployment group of AppConfig targets, such as applications in a beta or a prod environment. So we give our environment a name, can add an optional description. Now, what we want to do is expand monitors and here is where we select that IAM role that was created, CloudWatchAlarmDiscoveryRole. Now really, if we want CloudWatch alarm associated with our environment, we have to add an alarm. We can create a specific alarm. I'm just going to add an alarm just to save some time, then we can go ahead and create the environment. Now, when we created the environment, we were on the Environment tab. Let's go back to our application, so there's an environment. We could certainly create more environments, but there are also configuration profiles and feature flags. Let's select Create. So feature flags help you release features quickly and safely. It's basically an on/off switch for features, and we can create our own, but there are feature flags that come out of the box. Let's just select that. Alright, let's give our feature a name and create. Now, what we want to do is add a new flag, so we select Add new flag, we give our flag a name, and the key will auto populate. Now, here's a key point that doesn't really jump out at you, but right now, this flag is toggled off. So if we wanted to, and I'll do that, it's now on, and it's that simple with features and deploying them. Let's check out attributes. And so what we can do here is set up attributes based on string, numeric, Boolean, or arrays. Key part of this is this toggle here, and this will specify whether the flag is enabled or disabled, so I'll just turn that on. We could add some optional attributes, but I'm just going to add this flag. Alright, so now we have a new flag in our configuration. Next step is to create a deployment strategy. So let's go back to the main AppConfig page, there's our Deployment Strategies tab, select that, and this kind of looks familiar. Again, there's a similarity to CodeDeploy. Do we want to deploy all at once? 50% at a time? There's a canary type deployment, and let's just go with all at once. Give our deployment a name, an optional description, deployment type. Let's go with linear percentage of the deployment at a time. How long do we want it to take? Bake time, specify the amount of time to wait for completing the deployment, and let's just go ahead and create. Note that we could still go back to other deployment options strategies if we wanted to, but here's the one we created. Last, but not least, let's see about deploying a configuration. We don't have an application to deploy to, but we just check it



out. Let's go to our application. Alright, now we want to go to Environments, View details, Start the deployment, and we choose a configuration. We only set up one. Here, we would plug in a configuration version to deploy deployment strategy. We set up one, but there were three others we could use, and then we would start our deployment. So that is a look at how to set up AppConfig to begin deploying configurations to our applications. Alright, let's take a look at some exam tips. What hosts can house the apps that AppConfig can deploy to? Well, it's EC2 instances, AWS LAMDA, containers, mobile apps, and IoT devices. What's the difference between AppConfig and CodeDeploy? I think this is just really important to understand. AppConfig is deploying configuration changes to existing apps. CodeDeploy is deploying those apps. So AppConfig is focused on configuration, think of environment variables, feature flags. Do you need to know where AppConfig fits in the pipeline? Well, yeah, but with AppConfig, we can go straight to our application, it sort of circumvents the pipeline a bit, but certainly a part of our pipeline from a conceptual standpoint. What else do we need to know about AppConfig? Well, just remember there's use cases and benefits that we touched on in the lesson, and again, certainly understand the difference between AppConfig and CodeDeploy. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

AWS Control Tower

Hey Cloud Gurus, welcome to this lesson on AWS Control Tower. So what is Control Tower? Well, Control Tower provides a way to set up and govern in AWS multi-account environment. There's a term again, multi-account. We've already talked about deploying to multi-accounts. Well, we're going to have to be able to govern those environments and that's where Control Tower comes in, and Control Tower works with several other services, including and most importantly, AWS Organizations. So in this lesson, we'll take an overview look at Control Tower, talk a bit about how it works, and this is another service where we'll gain something from setting it up, so we'll also do a demo on creating a landing zone for a Control Tower. We'll talk about landing zones as well. And finally, wrap up with some exam tips. So with that said, let's get into Control Tower. Alright, what is Control Tower and why do we need it? Well, again, when we start deploying applications to multi-accounts and that is a big focus of the latest version of the DevOps exam, things can get a little complicated and we need a way to govern our environment, and that's where Control Tower comes into play. It's important to understand that Control Tower is tightly integrated with organizations. So how do we manage multiple accounts? Well, think about what Organizations does with the organizational units and those service control policies. Control Tower is an extension of organizations. So, I always like to use the term sort of wrapper. There's a bit more to it than it just being a wrapper for organizations, but it inherits all that good stuff from organizations, such as organizational units, and it provides additional control in multi-account



environments. So let's see what that looks like. How does this work? Here's our Organizations again, so yeah, they're very tightly coupled. So let's say we have a multi-account environment that we're deploying applications to. With Control Tower, we can create a well-architected multi-account baseline, and we're calling that a landing zone, and that landing zone is going to impose security and compliance best practices, so we're dealing with all of these different parts, different accounts with one overriding sort of compliance framework, and that landing zone again, going back to that coupling with organizations, holds all of our organizational units, our accounts, our users, and any other resources we want to subject to our landing zone compliance regulations. In the landing zone, when we create a landing zone, it will actually create organizational units and accounts for us based on best practices. These OUs are within our landing zone. This security OU has an audit and a log archive account. Then there's a sandbox OU which have a dev and a test account and a root OU with a management account, prod OU with a prod account. So again, the landing zones are well-architected baseline with all of these OUs built using compliance best practices and also security. Just think about the administrative overhead, the burden of that, that this takes from us. And the thing to understand about this is that okay, we have this log archive account, well, we have multiple accounts, we have multiple types of logs from all these different accounts, well, guess what? It's all consolidated here. So that's really a great feature of Control Tower in the landing zone. Now, with our prod account, we want to be more careful with what we're doing, and Control Tower provides guardrails, two types, one being preventative guardrails, and with those preventative guard rails, we can disallow API actions using service control policies. The other type of guardrails are detective guardrails, and they're used for governance and compliance. They work with Lambda functions and AWS config. So that's a look at AWS Control Tower, how it works, and why we need it for these multi-account environments, and let's check out a demo and just go ahead and solidify what we've learned here by setting up a Control Tower landing zone. Alright, at the Management Console, specifically AWS Control Tower, and we want to set up a landing zone, so I'll click that. A lot of good information here, necessary permissions to go govern all the OUs and their accounts within your organization's infrastructure, and this is all good information to read. Control Tower, by default, doesn't extend governance to your existing OUs and accounts. You'll see a landing zone set up in parallel to your existing organizations. When you bring an existing organization into AWS Control Tower, it's called registering. The landing zone is set up for us, but we have to register the OUs we want within that landing zone. Home region. I'm going to keep that as-is. We can deny access to specific regions, and I'm going to leave that not enabled. We could also set up additional regions to govern, and I'll just leave it as just us-east-1 and click Next. Alright, on this page, we have our foundational OU, and that's going to be that security OU, which has that audit account and log archive account. Alright additional OU, and that's going to be that sandbox OU we saw in the diagram, and just to be clear though, that in the



diagram, we saw a dev test account that's not set up for us, but if we want that, we could certainly go ahead and create that. So I'll click Next. Alright, and that brings us to the management account, provides billing and management of your accounts, relies on your existing AWS account email address. There's that log archive account. We could use an existing or create a new account, provide an email address for that. Then we have our audit account. Now, key point, log archive account, email address, and the audit account need to be different. Then we can go ahead and click Next. One last page where we can configure CloudTrail and encryption. CloudTrail delivers logged information to a specific S3 bucket. File path contains the org ID as a prefix, so I'm leaving it enabled. We can set up some retention that's optional, and we can set up KMS encryption. I'm just going to click Next, and then we can review everything. Looks a lot like reviewing a CloudFormation stack deployment. We just have to check this box understanding the permissions Control Tower we'll use, enforce on my behalf, understand the guidance on the use of Control Tower and underlying resources, and then we can set up our landing zone. Alright. Notice that this takes a while to set up saying 58 minutes, not going to wait that long. But let's check some things out. Control tower is setting up the following two organizational units, three shared accounts, native cloud directory with pre-configured groups and single sign-on access, 20 preventive controls, 3 detective controls. Notice that sandbox and security, there's a hierarchy here, and the root is the parent for these OUs. So there's a lot going on here, but ultimately, again, Control Tower is integrated with AWS Organizations and it enables us to easily set up security and governance for our multi-account environments. Alright, let's check out a few exam tips for Control Tower. Control Tower is an extension of AWS Organizations, and if you remember that and just remember the concepts of organizations, you'll be in good shape. When you create a landing zone, if you do not have an organization, Control Tower will create one for you. What are the two types of guardrails, preventative and detective. And the key component really is the landing zone. It's a well architected multi-account baseline that enforces best practices in our organizations, best practices such as security and compliance, and it's really the platform by which you can govern your multi-account environments. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

AWS Control Tower Migration

Hey Cloud Gurus. Welcome to this lesson, AWS Control Tower Migration. Now we've talked a lot about AWS Organizations and also Control Tower, and we know by now there's a lot of overlap between these services. So in this lesson, we'll talk about the differences between the two services, why you would choose one over the other, and we'll talk about migrating accounts to Control Tower if you've made the decision to use Control Tower. Alright, AWS Organizations vs Control



Tower, and let's talk about reasons why we would use AWS Organizations. It's reliable and it's pretty easy to use. You can integrate with AWS Single Sign-On and just some bottom line information, use AWS Organizations if you already have multiple accounts, you don't intend to have more than 50 accounts, so Organizations doesn't scale as well as Control Tower and if you use infrastructure as code for development. Organizations does integrate well with infrastructure as code. Now, what about Control Tower? We can initialize new AWS accounts with preset infrastructure, and that is pretty powerful. Now, on the downside, Control Tower lacks flexibility after accounts are provisioned. So when would we use Control Tower? When we plan to impose a lot of restrictions on every sub account, and when we don't need to do a lot of differentiated or complicated provisioning in different accounts, and that goes back to Organizations is a little bit easier to use. Alright, but let's say we've made the choice, and we want to go with Control Tower. Let's talk about moving from Organizations to Control Tower, and Control Tower extends what we can already do with AWS Organizations, and this really cuts right to the chase. We can apply guardrails for security, configuring accounts, and that pre-configuring of accounts is really powerful, monitoring activity in Control Tower, faster deployment times, and we can automate many of the capabilities of AWS Organizations. Now, let's step through migrating an account to Control Tower, and we certainly don't need to memorize this process, just be aware of it and a general idea of what's going on. Alright, so step one, our AWS account leaves the organization that it is in. Step two, the account becomes a standalone account. So at this point, it doesn't belong to any organization, and think about that, the ramifications. Governance and billing are managed independently by account administrators. So we have this free agent account hanging out there. Our target organization sent an invitation for the account to join its organization. The standalone account accepts the invitation and becomes a member of the target organization. Now notice the root organizational unit. Our account is enrolled in Control Tower. Watch this. It is moved to a registered organizational unit, and at this point, the account is under the administration of Control Tower in this new organization. So in this lesson, we talked a little bit about the differences between AWS Organizations and Control Tower, and the advantages of using each and when you would use each, and if you make a decision to move an organization to Control Tower, this is the process that you would go through. So just understand the key differences between these two services and really the fact that you can migrate accounts to Control Tower fairly easily, and you should be in good shape for the exam. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Amazon Detective

Hey Cloud Gurus. Welcome to this lesson on Amazon Detective. In this lesson, we'll take an overview look at Amazon Detective, going to talk about how



it works, and then we'll get a little bit hands on to solidify our knowledge with a demo. We'll come back from the demo and talk about Detective at scale, and then we'll finish up with some exam tips. So let's go ahead and get started with the overview and find out a little bit about Amazon Detective. Alright, what is Amazon Detective? Well, Detective helps you analyze, investigate, and quickly identify the root cause of security findings or suspicious activities. Now, we have a multitude of security services in AWS GuardDuty and SecurityHub and some of these services can notify us of issues. Some, such as Security Hub, also recommend changes. Now Detective, on the other hand, we have a security finding, but we need to dig deeper, and we need to find the root cause of this issue and fix it. How does this happen? Detective automatically collects log data from our AWS resources. It also collects login attempts and API calls, and again, back to the logs, CloudTrail logs, VPC flow logs, and it also ingests findings from GuardDuty and SecurityHub, and Detective uses machine learning statistical analysis and graph theory to assist in security investigations based on all of this information it has gathered, and a key point to remember, Detective can access and analyze data up to a year back in time. Alright, let's dig a little bit deeper on how this works. So again, Detective is gathering log data, CloudTrail logs, EKS audit logs, VPC flow logs. Detective automatically extracts time-based events, such as login attempts, API calls, network traffic, from these logs. It also ingests findings detected by GuardDuty, and we'll see when we work with Detective in the demo, we have to have GuardDuty enabled. In fact, you have to enable GuardDuty 2 days before you can even start Detective. In addition to GuardDuty, it also integrates with Security Hub. So all of these services are sort of integrated and go hand-in-hand, it's just important to remember what the purpose of each one is. Now, as far as graphing, Detective creates a unified interactive view of your resource behaviors and the interactions between them over time in graphical format. So with Detective, we can rapidly investigate any activity that falls outside of the norm. We can identify patterns that may indicate a security issue, and it helps us understand all of the resources affected by a finding. Alright. Now, how does Detective work at scale, and a big focus of this DevOps Pro exam update is multi-account, multi-region. How does Detective handle this? And of course, we're going to have multiple accounts, that could be up to 1200 accounts. How do we integrate Detective with all of this? Think about that before I reveal the answer, and you probably have a good idea. Something we've certainly covered. AWS Organizations can be integrated with Detective similar to how Control Tower integrates with Detective. So any account can be delegated as an administrator account and can centrally manage the Detective behavior graph database. So, we're going to be collecting a lot of log data from our multiple accounts. Now, Control Tower is an extension of Organizations. Detective is not an extension of Organizations, but it does work closely and can be integrated with Organizations. Organizations can help us manage all of this. Now, this is an external link to the AWS documentation. I'd recommend checking that out. It's just a Detective behavior graph. Alright, let's do a demo where we get an overview look at Amazon Detective from the



Management Console. Alright, out at the AWS Management Console, here is Amazon Detective, but I'm going to go to GuardDuty first. Alright, so the point is to work with Detective, you must first enable GuardDuty, and I enabled GuardDuty about 3 weeks ago. You have to have GuardDuty enabled for 48 hours and then if you enable Detective, it will work with GuardDuty, and specifically, your findings. I created a couple of findings here for the benefit of the demo. So I'll duplicate the tab and let's now go over to Detective. Alright, I have not yet enabled Detective. I'll need to click the Get started button, take a look at the pricing information. Please note you have a 30-day free trial. If you're doing this on your own, certainly set some kind of alarm for yourself. I'd suggest just watching the video is sufficient. For exam purposes, just know what Detective does and the services it can interact with, but let's go ahead and get started. I need to enter an account number, delegate administrators, grant all the permissions to administer Detective, and Detective will be enabled for that account and then I can click Delegate, and we can just go ahead and enable Detective. So I'm going to go ahead and do that. Alright, successfully enabled. We have a 30-day free trial, so let's just check this out a little bit. This is a pretty robust service, certainly worth checking out on your own. I'm just going to remove these notifications. So we initially come to account management. If we're using organizations, we can delegate some administration that way, or we can do it by invitation. But let's go to the Summary page, that's where our initial action is going to be, and we have four panels, finding groups, roles and users with most API call volume, EC2 instances, traffic volume, and clusters, and also some geolocation information. Alright, let's talk about finding groups. Finding groups correlate entities and security findings to the same underlying activity and should be investigated together, and just notice in the top right, we can sort of edit our finding page, our finding groups, edit the start date to view 24 hours of data on the summary page. But let's back out. Our main page is going to be a little bit different now. Alright, investigate your findings, learn how to view your GuardDuty findings in Amazon Detective and explore your data. So start investigating. Let's go over to GuardDuty. Alright, this brute force SSHBruteForce looks interesting. Just click on that, gives an overview, bunch of information here, but most importantly, investigate with Detective. So let's click that and click this big long number here, which is the finding ID. Scan that real quick. Note that they, they match. Alright, one last thing I want to show in this sort of overview demo. So each of our findings in GuardDuty have a severity of low. Now, if we're using Security Hub, remember, it's the hub of our findings from different services including GuardDuty, so we have two findings of low severity. Let's go here and product, I think we just have to sort the other way. There we go. There are our two findings. So I'll click on one of those just to show how this is all linked together, click on it. Here's our high-level information, but notice we have the option to investigate in Detective. So first of all, when we enable Detective, GuardDuty has to have been enabled at least 48 hours earlier. GuardDuty is closely linked with Detective. We can view our findings from GuardDuty Investigate over in Detective. We can also investigate our findings



that show up in Security Hub as well in Detective. And to wrap up on Detective, let's go back to our summary. This takes a little while to populate, just enabled it a few minutes ago, but notice starting to get some information here. Roles and users with the most API call volume, ServiceRoleForSystemsManager, ServiceRoleForElasticDisasterRecovery, and ServiceRoleForNetworkFirewall. There's a lot going on in Detective, but just wanted to get an idea of from a high level of how you use it and how it interacts with other services such as GuardDuty and Security Hub. Alright, let's check out some exam tips. What log files does Detective investigate? Cloud trail logs, VPC flow logs, EKS audit logs. What security devices or services does Detective interact with? GuardDuty and also Security Hub. How do we implement Detective at scale? As you might expect, we integrate with AWS Organizations for our multiple accounts in multiple regions, so we get all that good stuff with AWS Organizations to help us use Amazon Detective at scale. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

Amazon Athena Overview

Hey Cloud Gurus, welcome to this lesson, AWS Amazon Athena Overview, and Athena is a service that really has one clear-cut task that makes it stand out from other services, so a service level understanding of Athena will service well on the exam. So let's check out Amazon Athena. Alright, what is Amazon Athena? Athena is a serverless, interactive, analytic service built on open source frameworks supporting open table and file formats. Now, on the surface, there's a lot going on here, but if you know one thing about Athena, know this. Athena is an interactive query service that makes it easy to analyze data directly in Amazon S3 using standard SQL. Long story short, we can query objects in S3 using SQL commands with Athena. If you forget everything else, know that about Athena, but as you can see, it can do a lot more, but it's all in the same realm. It provides a simplified, flexible way to analyze petabytes of data, analyze data or build applications from an S3 data lake, and 30 data sources, including on-premises data sources or other cloud systems using SQL or Python. So this is a SQL-based tool, and certainly, check out all of the different possible inputs on the left, probably the most well-known and prominent, at the bottom S3. Alright, let's talk about some use cases for Athena. Well, first and foremost, run queries on S3, also on-premises or other clouds. Prepare data for ML models, machine learning, so let's just focus on S3. We have data in S3, but we need to format it in a way that is friendly for machine learning, and we can do that with Athena. Perform multi-cloud analytics. Another use case build distributed big data reconciliation engines. So what does some of these big words mean, reconciliation? Think data validation, so we're using Athena to validate large amounts of data for this big data engine. Now what Athena is probably most famous for, if that's possible to be famous for querying data, it is querying S3 data. So, the steps we create in



Athena database, we create a table and we query S3 to populate that table in Athena. Let's check that out in the Management Console, just get a look at it and see how it works, and it's a really cool service, and again, if you remember one thing about Athena, we can query S3 objects with Athena using SQL commands. Alright, so I'm in the Management Console, Amazon Athena. Note our two options here, use Query Editor to analyze data on S3, on-prem, or on the clouds. Notice an association with Apache Spark, but let's stick with S3, Launch Query Editor. We have a very important note here. Before you run your first query, you need to set up a query result location in S3, so let's click Edit settings. Now, if we know the path of our bucket, we can enter it here. Let's just browse, and I'm looking for a bucket that ends in 2024. That's it. I've been using that for the course. Choose that one, note on lifecycle roles, assign bucket owner full, control over query results, but this is all within my account, and I can just click Save. Now from here, I'll click on Editor and this is our Query Editor. So right here, we would plug in our SQL and run it. Once we plug it in, this won't be grayed out, we run it. So our next step is to create a database named my database. Notice, our run is live now, so I'll click that. Alright, completed query successful. I refresh this left pane and notice I do indeed have a new database, and I select it. Alright, notice we have a plus sign here. So for a new query, I click the plus sign, and we created a database with this first query. We could close that, but I'll just leave it hanging around. Alright, here's our SQL for creating a table and notice it's based on CloudFront logs. I'll include this query in the lesson material. So basically, if you look at the fields here, again, it's related to CloudFront logs, and then there's some Regex going on, which is breaking out operating system browser and browser version info. Don't worry if this is a bit complicated on the exam. You're not really going to be tested on complicated SQL if at all, and notice where this is coming from, Athena examples by region, so this is just an example, but with that said, this needs edited, us-east-1. So this query is accessing CloudFront logs from this location. This is an AWS sample location, it's parsing and formatting. Let's run it. That was quick. Query successful. Notice we have a table, CloudFront logs. We have a formatted table corresponding diagonally up here to our SQL command. Alright, so just so we're sure we understand what's going on here, basically, we had this sample S3 bucket provided by AWS. We're using Athena to query that data. With this query, this SQL, we've created a table and we populate our table with that data, the table being, of course, in Athena. Okay, so we have a table with data in it. Can we query that data? Well, let me open another tab, plug in a command, a select statement. Select from CloudFront logs, our Athena table we just built from data provided by S3, and we're grouping by operating system. Let's run that. Alright, so that took a whole 3 and almost a half second, so pretty darn big table. So we're grouping by OS, Mac, Linux, Windows, iOS, Android, OS X, and remember what we're doing. We're getting a count by OS of all the rows in our table. There's our count. Yeah, and I don't know about you, but I think that's pretty cool. So we grab data from an S3 bucket, parsed it, and populated our Athena table, and now we're able to query that data. So I think that's pretty darn powerful, but this is one of its



very main functions, querying data from S3 using SQL commands. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Configuration Management and Infrastructure as Code Review

Configuration Management and Infrastructure as Code Scenarios

Hey Cloud Gurus, welcome. In this lesson, Configuration Management and Infrastructure as Code Scenarios, we're going to take sort of a section review and look at some scenarios related to this domain, configuration management and infrastructure as code. So we'll just cover the basics here of what we're looking for and then we'll go over some scenarios, so let's check out the basics first. And this is geared toward presenting us some scenarios and working with visualization. Read the scenario, try to visualize it, also sketch it out, what you visualize, sketch that out on a Notepad and then we'll reveal the solution and go over it. Now remember, on the exam, we have roughly 2 minutes per question on the exam, and you really don't have time to sketch out every question, but certainly some of the harder ones, you can sketch out and it's helpful. And what I often do on the exam is I'll skim and read through a scenario question quickly and if it seems pretty hard to me, I'll mark it for later. I'll go through the exam, and oftentimes, you'll run into scenarios that might jog your memory on another question that you've marked for review, and almost always, those questions I marked for review seem easier after I've gone through the exam, get my confidence up, and then I have time to focus on the harder questions near the end. So really practice your visualization and just keep in mind we're moving pretty fast. Alright, let's check out some scenarios. Alright, our company has multiple Elastic Beanstalk environments and different deployment requirements for each. What if our requirement is minimal downtime and easy rollback? What kind of solution are we going to use? We're going to use blue/green deployments, and it meets these requirements. Now, a caveat, what if the requirement is absolutely no downtime with full capacity during deployment? Think about that. Alright, so we have rolling deployment with additional batch, so that additional batch, we're working on our new deployment in this additional batch, but our regular deployment is untouched while we're deploying this additional batch. Now, the downside is we're doubling our capacity, but it's for a short time so it's not going to be a real big financial hit. Let's check out another scenario. We have developed a new version of Lambda application and it's ready to be deployed. The deployment should not cause any downtime, and we need a quick rollback, what steps can we take. So trying to get that mental picture of this



scenario and what we're doing with Lambda and how we can deploy to meet these requirements, and certainly pause the video if you need to, but let's see what we have here. Alright, so we have a Lambda function version 1. We're going to make a version 2. Ah, we'll create an alias named prod. Initially, that's going to be pointing to our version 1. So we prepare version 2, we test it, it's ready to go. What do we do? We simply point our alias to version 2. So we create a prod alias and point it to the new version. Now, if problems arise, we can just point back to this version 1, so that's our quick rollback. And at some point when you're certain that you're stable, well, you can just get rid of version 1 on your prod server. Alright, let's check out another scenario. Okay. We have an autoscaling group and a CloudFormation stack, so just visualize that and sketch it out. When our autoscaling group scales out, the old instances should only be terminated when the new instances are fully operational. So that's our scenario, and you have to derive these clues from the scenario. Old instance is only terminated when new instances are fully operational. How do we work this out? Alright, so we have our autoscaling group in our CloudFormation stack, and it's actually going to be a setting in our template. So for our autoscaling group in CloudFormation, we have this attribute, autoscaling, replacing, update, and we set the WillReplace property to true and this will satisfy this requirement. Old instances should only be terminated when new instances are fully operational, and this is one where hey, if this is a little sketchy for you, just check out the documentation, AutoscalingReplacingUpdate. These are things that you can't memorize everything in the CloudFormation documentation, but this is something you want to know. Alright, one more scenario. We want to modularize our stacks, CloudFormation stacks, in a way that we can manage stacks from a single point. That's a pretty good hint given what we know from some of the lessons in this section of the course. Manage stacks from a single point, so think about that, sketch it out, think about your solution. I'll give one hint, manage stacks from a single point kind of sounds like, I don't know, a root stack maybe. Lets check it out. Yep. Well, in this case, root stack, parent stack, children's stack, same concept. So we're talking about nested stacks and the modularization. So we could have a parent stack, maybe this one deals with S3 buckets, maybe this one deals with autoscaling groups and these are reusable across different stacks, and the great thing about that is if something needs changed in these templates, we can make the change in one place rather than 50. Alright, let's take a look from a different angle. What if we want to modularize, but manage stacks separately? Alright, visualize, sketch out, and think about your solution. Ah, so we have two stacks. Now, we want them sharing information, but we want to manage them as separate entities. And by now, probably know where this is going, good old cross-stack references. So we can share information across these stacks, and that in a sense is modularizing our templates, but we manage StackA separately from StackB, and that's really a good way to think of nested stacks and cross-stack references. Nested stacks we manage in one place, cross-stack references we manage in multiple places, both of them in their own way modularize our code. Alright, so that's a look at some scenarios for



configuration management and infrastructure as code. Congratulations Cloud Gurus on completing this section. I'll see you in the next section.

DevOps Conversations for Configuration Management and Infrastructure as Code

Hey Cloud Gurus. Welcome to this lesson, DevOps Conversations for Configuration Management and Infrastructure as Code. So this is another lesson where we're going to look in on a conversation and try to learn a little bit along the way. This conversation is sort of an in-house interview between a manager and a DevOps engineer. The manager is asking some questions, gauging the engineer's knowledge to see how much more responsibility to give the engineer. So with that said, let's look in on this conversation and see what we can learn along the way. DevOps lead, you're doing a great job so far, so I'm thinking of upping your responsibilities. Let's have a question and answer session, so I can pinpoint how much more to give you. I don't know how rare this is, but I've had a couple of these in my career. DevOps lead goes on to say we're using CloudFormation to deploy our infrastructure and Elastic Beanstalk to deploy applications. Is there a way we can combine these? It may be one of your first tasks. Junior engineer. Yes, CloudFormation has a resource type for Elastic Beanstalk applications, so certainly CloudFormation and Elastic Beanstalk can be combined, CloudFormation for the infrastructure and, of course, Elastic Beanstalk to deploy those applications. Alright, DevOps lead. Well, that's a good start, but I have concerns about RDS. We often delete Elastic Beanstalk applications in our dev and test environments, and what the DevOps lead is inferring is that the database is tightly coupled. It's created in Elastic Beanstalk, so when the applications are deleted, the databases are deleted as well. The lead wants to retain the databases. How would you handle this? Junior engineer says, first, I'd launch an RDS database instance external to Elastic Beanstalk, so it's not part of that Elastic Beanstalk app. Then I'd modify the inbound rules of the database instance security group to allow access by instances within the Elastic Beanstalk environment. So reading between the lines, when the Elastic Beanstalk is deleted, the database is retained because it was not part of the Elastic Beanstalk deployment. DevOps lead goes on to say let's talk about the CDK a bit. Can I use the CDK, instead of CloudFormation? And what's so great about the CDK anyway? Junior engineer, sure, the CDK could be used instead. It resolves to CloudFormation under the hood. Big advantage you can write much less code to get the same results because it uses higher level powerful languages such as Java and .NET. And what we want to remember from this is when we deploy CDK code, it does resolve to CloudFormation under the hood and you'll end up with a CloudFormation stack. DevOps lead gets a little tricky here, wants to check on the junior DevOps engineers context switching here. With CodeDeploy, what are the deployment targets that we can use? Junior engineers on top of it. You can deploy to EC2 or on-premises servers, ECS, and Lambda. And I suppose the point



here is just to keep your mind nimble. Certainly going to have a lot of context switching on the exam. DevOps lead back to configuration management and IAC conversation. Are you comfortable writing CDK code? And what step could you take to avoid reinventing the wheel in the CDK? Sure. I'm pretty handy with C#, Java, and Python. And of course, those are some, if not all of the languages you can use with the CDK. Junior engineer goes on to say I'd check the AWS Construct Library to see if there are constructs I could use for my task before writing any code. The point is, there's that AWS Construct Library. Always think code reuse, check that, see what you can reuse. You don't need to reinvent the wheel. DevOps lead says, "What about serverless apps? Can you do that in the CDK? What are your options?" Junior engineer answered, "Yeah, we can create several apps with the CDK, but points out the other option being SAM, AWS Serverless Application Model. It's similar to using the CDK instead of CloudFormation, it depends on your comfort level with writing high-level code versus templates. SAM uses templates like CloudFormation, but it's just for serverless." Alright, we come to the end of that conversation, and these conversations are really to focus on key topics and fundamental understanding of services that can really help you on the exam, oftentimes, eliminating incorrect answers without even having to take the time or the brain power to read them. You pick out key words or key terms. If the answer does not contain those, eliminate them, don't waste another second on them. So with that, I'll end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

Supporting Services for Configuration Management and IaC

Hey Cloud Gurus. Welcome to this lesson, Supporting Services for Configuration Management and Infrastructure as Code. So these are some services, they may not figure prominently in this domain, configuration management and infrastructure as code, but you may run into them, and it's a good idea to know what they do. The services we are going to talk about are serverless application repository, AWS app to container and AWS Copilot, and we will finish up the lesson with some exam tips. So let's first start talking about serverless application repository, and it is a managed repository. There's that keyword, AWS managed for serverless applications. Store and share reusable applications and easily assemble and deploy serverless architectures. So, let's say we have some SAM templates that we've used. We want to reuse these templates, and we can store these in serverless application repository. And of course, when we talk about serverless and AWS, we think Lambda functions and then some additional things like event sources and other resources. This can all be packaged up and stored in the serverless application repository. Ultimately, SAM templates are just like CloudFormation templates, except for they're for serverless, and the end product we would deploy as CloudFormation, but it all starts with the servers application repository. And the idea is we can use pre-built applications



within serverless application repository, those applications that we want to reuse, and we're going to reduce duplicated work. We're not going to reinvent the wheel. We're going to make our team members aware of what's in this repository and reuse that code, and this code follows best practices, so we'll get a better product out to market faster. So again, probably not a huge part of the DevOps exam, but certainly want to know what it is and what we can do with it. Let's talk about App2Container. So if we just slow down and think this through app to container, we have a pretty decent idea what's going on here. So let's say we have on-premises legacy applications maybe in Java or .NET, and we want to move those to AWS, and we're seeing some container services here, app container and then the to part is App2Container. It's going to take these applications and containerize them. So how can we get a non-standard legacy application into AWS? Well, we containerize it. And again, think about that name, App2Container. So specifically, what App2Container is, it's a command-line tool for migrating and modernizing Java and .NET web applications into container format, and ultimately, once the applications are containerized, App2Container can deploy the app to AWS. Now using ECS as an example, it would store the container image in ECR and deploy to ECS, so some maybe key terms here on-premises legacy applications or non-standard legacy applications. How do we get that into AWS? Containerize it, we may have known that already, but AWS now has a service that's going to help us with that. Alright, let's talk about Copilot. AWS Copilot, a command-line interface that can be used to quickly launch and manage containerized applications on AWS, so we launch from Copilot to such services as ECS, Fargate, and App Runner. It kind of sounds like App2Container in a way, but think about App2Container containerizing legacy on-premises applications and think about Copilot. It is also command-line interface working in AWS to quickly launch and manage containerized apps, and command-line interface, anything we can do from command line, we can certainly script. And if we need to distinguish further between App2Container and Copilot, App2Container is specific for Java and .NET applications. Alright, so that's some of the supporting services for the configuration management and infrastructure as code domain. Let's look at some exam tips. What is being stored in the serverless application repository, SAM templates, and just remember the relationship between SAM and CloudFormation. It's going to look a lot alike. Templates are very much the same, except SAM is for serverless. What key terms would help identify App2Container as part of a solution? On-premises applications, Java, or .NET, and maybe an additional one legacy applications. Where do you work with AWS Copilot? From the AWS command line. So with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.